



ФАЙЛОВАЯ СИСТЕМА «JSON FS»

ДОКУМЕНТАЦИЯ

Автор: Vecsai

Версия: 2.0.0

Год: 2026

GitHub: github.com/Vecsai/JSONFS

СОДЕРЖАНИЕ

1 ПРОГРАММНАЯ ДОКУМЕНТАЦИЯ	4
1.1 Описание программы.....	4
1.2 Руководство оператора	6
2 ПРАКТИЧЕСКАЯ ЧАСТЬ.....	11
2.1 Описание структуры.....	11
2.2 Защита от несанкционированного доступа	15
2.3 Демонстрация функциональных возможностей.....	22
2.4 Аварийные ситуации	38
2.5 Эргономичность программного продукта.....	42
ПРИЛОЖЕНИЯ	49
ПРИЛОЖЕНИЕ А Модульная структура JSON FS	49
ПРИЛОЖЕНИЕ Б Сравнение JSON-файлов до и после редактирования	50
ПРИЛОЖЕНИЕ В Листинг программы	51

1 ПРОГРАММНАЯ ДОКУМЕНТАЦИЯ

1.1 Описание программы

JSON FS — это специализированная файловая система, разработанная с использованием технологии **FUSE** (Filesystem in Userspace), которая позволяет пользователю работать с JSON-файлом так, как если бы это была обычная файловая система в Linux.

1.1.1 Библиотека FUSE

FUSE — это механизм в Linux, который позволяет создавать файловые системы, работающие в пространстве пользователя, а не в ядре операционной системы. Это значит, что разработчику не нужно писать сложный код ядра, достаточно написать обычное приложение, которое будет обрабатывать запросы к файловой системе.

FUSE перехватывает системные вызовы, связанные с файлами (например, открытие, чтение, запись, создание каталогов), и передаёт их в программу, которая реализует логику файловой системы. Эта программа отвечает, как именно обрабатывать эти запросы и что возвращать.

Таким образом, с помощью FUSE можно создавать виртуальные файловые системы, которые отображают данные из любых источников — удалённых серверов, баз данных, API и, в нашем случае, из JSON-файла.

1.1.2 Принцип работы JSONFS

В JSONFS JSON-файл воспринимается как дерево каталогов и файлов:

- Объекты JSON (то есть наборы ключей и значений, где значения — это другие объекты или примитивы) отображаются как каталоги (папки).
- Параметры с примитивными значениями (строки, числа, булевы значения и т.п.) отображаются как файлы.

Это позволяет пользователю использовать привычные команды Linux для работы с JSON-структурой:

- **mkdir** — создать новый раздел (объект JSON).
- **cat** — прочитать значение параметра (содержимое файла).
- **echo** или **cat > filename** — изменить значение параметра.
- **rm (rmdir)** — удалить параметр или раздел.

Все эти операции напрямую изменяют структуру JSON-файла, при этом пользователь не взаимодействует с самим файлом напрямую, а работает через файловую систему.

1.1.3 Преимущества такого подхода

- Интуитивность: Пользователю не нужно знать синтаксис JSON или использовать специализированные редакторы — достаточно базовых знаний командной строки.
- Удобство: Можно использовать любые стандартные утилиты Linux для обработки данных.
- Гибкость: Поддерживается вложенность JSON с произвольной глубиной, что позволяет работать с очень сложными структурами.
- Безопасность: Все изменения проходят через контролируемый интерфейс, что снижает риск повреждения данных.
- Автоматизация: Изменения автоматически сохраняются в исходный JSON-файл, исключая необходимость ручного сохранения.

1.1.4 Ключевые компоненты и архитектура

- libjansson — библиотека для парсинга (процесса преобразования одного формата данных в другой, в более читаемый формат), создания и изменения JSON-структур на языке C. Она отвечает за загрузку JSON-файла в память, доступ к элементам и сохранение изменений.
- libfuse — библиотека, реализующая интерфейс файловой системы в пользовательском пространстве. Она принимает запросы от ядра Linux и передаёт их программе JSONFS, которая обрабатывает их согласно логике работы с JSON.

Основные функции FUSE, реализованные в JSON FS, включают:

- getattr — получение атрибутов файла или каталога (тип, размер, права).
- readdir — чтение содержимого каталога (список ключей объекта JSON).
- mkdir — создание нового объекта JSON (каталога).
- create — создание нового параметра (файла).
- open, read, write — операции открытия, чтения и записи файлов (параметров JSON).
- unlink, rmdir — удаление параметров и разделов.
- rename — переименование элементов JSON.
- truncate — обнуление значения параметра.
- flush, release — сохранение данных и освобождение ресурсов.
- init, destroy — инициализация и очистка при монтировании и размонтировании.

1.1.5 Особенности реализации JSON FS

- При монтировании JSON-файл загружается целиком в память, что ускоряет операции и упрощает работу с данными.
- Все изменения сначала вносятся в память, а затем периодически сохраняются обратно в файл, что повышает производительность и снижает риск повреждения.
- При завершении работы (например, при нажатии Ctrl+C) программа автоматически сохраняет все изменения, гарантируя сохранность данных.
- Для предотвращения одновременного доступа и повреждения файла используется механизм блокировки при записи.
- Ключи JSON, начинающиеся с точки (.), считаются скрытыми и не отображаются в файловой системе, что позволяет хранить служебные данные.
- При записи новых значений программа автоматически распознаёт тип данных (число, булево, null или строка) и сохраняет их корректно в JSON.

1.2 Руководство оператора

1.2.1 Назначение программы

Файловая система JSON FS предназначена для удобного и интуитивно понятного взаимодействия с данными в формате JSON. Программа позволяет монтировать JSON-файл как виртуальную файловую систему в операционной системе Linux, предоставляя доступ к разделам и параметрам JSON в виде каталогов и файлов соответственно.

Основное назначение программы - предоставить пользователю возможность работать с JSON-структурами через стандартные файловые операции (создание, чтение, запись, удаление), используя привычные команды оболочки Linux (mkdir, cat, echo, rm и др.). Это упрощает управление сложными и вложенными JSON-данными, повышает удобство и снижает риск ошибок при редактировании.

JSON FS может применяться в различных сферах, где требуется динамическое управление структурированными данными, включая разработку программного обеспечения, администрирование систем, обработку конфигурационных файлов и обмен данными между сервисами.

1.2.2 Условия выполнения программы

Для корректной работы JSON FS необходимо соблюдение следующих условий:

1. Операционная система: Linux с поддержкой FUSE (Filesystem in Userspace).

2. Права доступа: Пользователь должен иметь права на чтение и запись JSON-файла, а также права на монтирование файловых систем с использованием FUSE.

3. Исходный файл: JSON-файл, содержащий корректно сформированную структуру данных в формате JSON.

4. Установленные библиотеки:

1) `libjansson` - для работы с JSON-структурами;

2) `libfuse` - для реализации файловой системы в пространстве пользователя.

5. Аппаратные требования: стандартное оборудование, поддерживающее работу ОС на ядре Linux и FUSE, без особых ограничений по ресурсам.

Перед запуском рекомендуется проверить целостность и корректность JSON-файла, а также убедиться в наличии необходимых прав доступа.

1.2.3 Сборка и компиляция программы

Для корректной работы и обновления файловой системы JSON FS оператору может потребоваться самостоятельно скомпилировать программу из исходных кодов. Ниже приведена подробная инструкция по сборке (процесс компиляции программы см. в приложении Г).

1.2.3.1 Порядок компиляции

1. Открыть терминал и перейти в каталог с исходным кодом.

2. Выполнить команду для компиляции: `gcc -o «название программы» «исходный файл для компиляции» -lfuse -ljansson -D_FILE_OFFSET_BITS=64`.

Пример: `gcc -o jsonfs11 jsonfs11.c -lfuse -ljansson -D_FILE_OFFSET_BITS=64`

Объяснение команды:

- **gcc** - вызов компилятора GNU для языка C, который выполняет препроцессирование, компиляцию, ассемблирование и линковку исходного файла.

- **-o jsonfs11** - опция задаёт имя выходного исполняемого файла. В данном случае скомпилированная программа будет называться `jsonfs11`.

- **jsonfs11.c** - исходный файл на языке C, который компилируется.

- **lfuse** - указание линковщику подключить библиотеку `libfuse`. Эта библиотека реализует интерфейс FUSE (Filesystem in Userspace), необходимый для создания пользовательской файловой системы.

- **-ljansson** - подключение библиотеки `libjansson`, которая обеспечивает работу с JSON-структурами (парсинг, создание, модификацию JSON).

- **-D_FILE_OFFSET_BITS=64** - директива препроцессору, которая определяет макрос `_FILE_OFFSET_BITS` со значением 64. Это заставляет использовать 64-битные

версии функций работы с файлами (например, для поддержки больших файлов свыше 2 ГБ), обеспечивая корректную работу с большими объёмами данных.

1.2.4 Выполнение программы

1.2.4.1 Запуск и монтирование файловой системы

Для запуска JSON FS необходимо выполнить команду монтирования, указав путь к JSON-файлу и точку монтирования:

jsonfs <путь_к_JSON_файлу> <точка_монтирования>

Пример: **./jsonfs /mnt/jsonfs test.json**

После успешного монтирования в каталоге /mnt/jsonfs будет доступна структура JSON-файла в виде каталогов и файлов.

2.2.4.2 Основные операции с файловой системой

- Создание разделов (каталогов): **mkdir /mnt/jsonfs/новый_раздел**. Создаёт новый объект (раздел) в JSON-структуре.
- Чтение параметров (файлов): **cat /mnt/jsonfs/раздел/параметр**. Отображает значение параметра JSON в текстовом виде.
- Изменение параметров: **echo "новое_значение" > /mnt/jsonfs/раздел/параметр** либо если вы создаете в самом каталоге **echo "Привет, мир!" > filename.txt** (можно и без кавычек: **echo "Привет, мир!" > file**). Изменяет значение параметра JSON. Поддерживаются строки, числа и булевы значения.
- Удаление разделов и параметров: **rmdir -r /mnt/jsonfs/раздел** и **rm /mnt/jsonfs/раздел/параметр**. Удаляет раздел или параметр из JSON-структуры.
- Переименование и перемещение: Используются стандартные команды **mv** для переименования и перемещения файлов и каталогов внутри монтированной файловой системы. Например, для переименования – **mv oldname.txt newname.txt**, а для перемещения – **mv myfile.txt mydir/**.
- показ список файлов и каталогов в текущей директории: используется командой **ls**.

2.2.4.3 Завершение работы

Для корректного завершения работы необходимо размонтировать файловую систему. Можно это сделать двумя способами:

- 1) **sudo umount /mnt/jsonfs**
- 2) **fusermount -u /mnt/jsonfs**

Это гарантирует сохранение всех изменений в исходном JSON-файле и освобождение ресурсов.

1.2.5 Сообщение оператору

В процессе работы JSON FS может выводить сообщения и уведомления. Подробнее об этом можно ознакомиться в «Таблица 2 – Сообщения программы при её некорректном выполнении».

Таблица 2 – Сообщения программы при её некорректном выполнении

Сообщение	Описание ситуации	Рекомендуемые действия оператора
Error: JSON file access denied	Отсутствуют права на чтение или запись JSON-файла	Проверить права доступа к файлу, запустить программу с нужными правами
Error loading JSON file: <текст>	Ошибка синтаксиса или повреждение JSON-файла	Проверить корректность JSON-файла, восстановить из резервной копии
Error saving JSON file	Ошибка при записи данных в JSON-файл	Проверить состояние диска, права доступа, наличие свободного места
File lock failed: <текст>	Не удалось установить блокировку файла	Убедиться, что JSON-файл не используется другим процессом
Operation not permitted (EPERM)	Попытка создать скрытый файл или выполнить операцию без прав	Изменить имя файла, получить необходимые права доступа
No such file or directory (ENOENT)	Обращение к несуществующему файлу или каталогу	Проверить правильность пути и наличие объекта
Is a directory (EISDIR)	Попытка выполнить операцию с файлом, который является каталогом	Проверить тип объекта и корректность операции
Not a directory (ENOTDIR)	Попытка обратиться к элементу как к каталогу, если он файлом	Проверить структуру и путь к объекту
Input/output error (EIO)	Общая ошибка ввода-вывода, например, при повреждении файла	Проверить состояние носителя, логи системы
Permission denied (EACCES)	Отказ в доступе из-за недостаточных прав	Проверить права пользователя и группы
Resource temporarily unavailable (EAGAIN)	Не удалось получить блокировку файла, ресурс занят	Повторить операцию позже или завершить процессы, блокирующие файл
Disk quota exceeded (EDQUOT)	Превышение квоты дискового пространства	Освободить место на диске или увеличить квоты
File name too long (ENAMETOOLONG)	Имя файла или каталога превышает максимально допустимую длину	Использовать более короткие имена
Read-only file system (EROFS)	Попытка записи в файловую систему, смонтированную в режиме только для чтения	Проверить режим монтирования, изменить при необходимости

При возникновении ошибок оператору рекомендуется внимательно изучить сообщение, проверить права доступа и целостность JSON-файла, а при необходимости восстановить данные из резервной копии или обратиться к системному администратору.

Памятка для оператора:

- Все сообщения об ошибках выводятся в консоль и/или системный журнал (syslog), что позволяет оператору своевременно реагировать на проблемы.
- При возникновении ошибок рекомендуется проверять права доступа, корректность JSON-файла, состояние носителя и логи системы.
- В случае повреждения JSON-файла следует использовать резервные копии для восстановления данных.
- При блокировках файла необходимо убедиться, что другие процессы не мешают работе с JSON FS.

2 ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Описание структуры

2.1.1 Общая структура и назначение

Разрабатываемый программный продукт представляет собой виртуальную файловую систему JSON FS, реализованную с использованием библиотеки FUSE (Filesystem in Userspace) и предназначенную для работы с данными, хранящимися в формате JSON. Основная цель данной системы - предоставить пользователю возможность взаимодействовать с JSON-документом как с обычной иерархической файловой системой, где каталоги соответствуют объектам JSON, а файлы - значениям различных типов (строки, числа, булевы значения и т.д.). Такая абстракция упрощает работу с JSON-структурами и позволяет использовать стандартные файловые операции для их изменения и просмотра.

2.1.2 Структура меню и интерфейс взаимодействия

Поскольку JSON FS – это файловая система, взаимодействие с ней осуществляется не через графический интерфейс или меню, а посредством стандартных системных вызовов и команд операционной системы. Пользователь работает с файловой системой через привычные команды создания, чтения, записи, удаления и переименования файлов и каталогов.

Основные операции, реализованные в системе, включают:

- Получение атрибутов файла или каталога (getattr) – возвращает информацию о типе объекта (файл или каталог), правах доступа, размере и времени модификации.
- Чтение содержимого каталога (readdir) – позволяет получить список файлов и подкаталогов в указанном каталоге.
- Создание каталога (mkdir) – создаёт новый каталог в структуре JSON.
- Удаление каталога (rmdir) – удаляет пустой каталог.
- Создание файла (create) – создаёт новый файл с пустым содержимым.
- Удаление файла (unlink) – удаляет файл из JSON-структуры.
- Открытие файла (open) – открывает файл для чтения или записи, с поддержкой блокировок при записи.
- Чтение файла (read) – возвращает содержимое файла, преобразуя JSON-значение в строку.
- Запись в файл (write) – записывает новые данные в файл, автоматически определяя тип значения (строка, число, булево значение или null).
- Переименование (rename) – перемещает или переименовывает файл или каталог.

- Обрезка файла (truncate) – поддерживает обрезку файла до нулевой длины.
- Синхронизация данных (flush) – принудительно сохраняет изменения в JSON-файл.
- Инициализация и завершение работы (init, destroy) – загружает JSON-файл при монтировании и освобождает ресурсы при размонтировании.

2.1.3 Модель и структура данных

В основе JSON FS лежит JSON-файл, который выступает в роли единой базы данных и хранит всю информацию о структуре и содержимом файловой системы. Формат JSON выбран из-за его простоты, гибкости и широкого распространения.

Структура JSON-файла в целом представлена в виде древовидной структуры файловой системы:

- Корневой объект - представляет корневой каталог файловой системы.
- Каталоги - представлены JSON-объектами, где каждый ключ - имя файла или подкаталога, а значение - либо вложенный объект (подкаталог), либо примитивное значение (файл).
- Файлы - ключи с примитивными значениями: строки, целые и вещественные числа, булевы значения (true/false) или null.

Каждый элемент JSON соответствует файлу или каталогу, что позволяет легко проводить навигацию по структуре с помощью путей, аналогичных файловой системе.

Таблица 3 – Структура JSON-файла

Название поля	Описание поля	Тип значения	Свойства
ключ	Имя файла или каталога	string	Уникально в пределах объекта
значение	Содержимое файла или каталог	object / string / int / float / bool / null	Если object - каталог, иначе файл

2.1.4 Перечень модулей и их назначение

Программный продукт структурирован в виде набора функциональных модулей, каждый из которых отвечает за определённый набор операций.

Таблица 4 – Перечень модулей и их назначение

Модули	Назначение
load_json	Загрузка JSON-файла с диска в память (корневой объект файловой системы).
clean_json	Рекурсивная очистка JSON-объекта от скрытых элементов (ключей, начинающихся с точки).
save_json	Сохраняет текущий JSON-объект в файл, предварительно очищая его от скрытых элементов.
log_operation	Логирование операций файловой системы в системный журнал (syslog).
jsonfs_init	Инициализация файловой системы при монтировании: проверка доступа и загрузка JSON.
jsonfs_destroy	Очистка ресурсов и освобождение памяти при размонтировании файловой системы.
sig_handler	Обработка сигналов SIGINT/SIGTERM для корректного завершения работы и сохранения данных.
get_json_by_path	Получение JSON-объекта по файловому пути ("/a/b/c") для навигации по структуре ФС.
jsonfs_getattr	Получение атрибутов файла или каталога (аналог stat) для FUSE.
jsonfs_readdir	Чтение содержимого каталога (список файлов и папок) для FUSE.
jsonfs_mkdir	Создание нового каталога в файловой системе (и в JSON-структуре).
jsonfs_create	Создание нового файла в файловой системе (и в JSON-структуре).
jsonfs_open	Открытие файла, проверка доступа и блокировка для записи (если нужно).
jsonfs_release	Закрытие файла, снятие блокировок, сохранение изменений.
jsonfs_read	Чтение данных из файла (извлечение содержимого из JSON-объекта).
jsonfs_rename	Переименование файла или каталога (перемещение значения в JSON-структуре).
jsonfs_write	Запись данных в файл (обновление содержимого JSON-объекта).
jsonfs_unlink	Удаление файла из файловой системы и из JSON-структуры.
jsonfs_rmdir	Удаление каталога (если он пуст) из файловой системы и из JSON-структуры.
jsonfs_truncate	Обрезка файла (поддерживается только обнуление содержимого).
jsonfs_flush	Принудительное сохранение данных при сбросе буфера.
jsonfs_oper	Структура, описывающая соответствие операций FUSE реализованным функциям драйвера.
main	Точка входа: обработка аргументов, установка обработчиков сигналов, запуск FUSE.

Для более наглядного и обобщённого представления структуры проекта см. приложение А, где приведена схема модульной структуры JSON FS.

2.1.5. Особенности реализации

К особенности реализации JSON FS можно отнести следующее:

- Используется библиотека FUSE версии 3.0 для реализации пользовательской файловой системы.
- Для работы с JSON применяется библиотека Jansson.
- Структура JSON-файла соответствует иерархии каталогов и файлов.
- При операциях записи происходит автоматическое преобразование типов данных (строки, числа, булевы значения, null).
- Поддерживаются блокировки файла JSON при записи для предотвращения повреждения данных.
- В системе реализована фильтрация скрытых файлов и каталогов (ключи, начинающиеся с точки, игнорируются).
- Все операции сопровождаются логированием через syslog.
- Обеспечена автоматическая синхронизация и сохранение данных при получении сигналов завершения работы.

2.1.6 Вывод описания структуры

Подводя общий вывод описания структуры, можно отметить, что разработанная файловая система JSON FS представляет собой комплексный программный продукт, основанный на использовании библиотеки FUSE для реализации пользовательской файловой системы и библиотеки Jansson для работы с JSON-данными. Анализ структуры программного продукта позволяет выделить ключевые компоненты, их функциональные особенности и взаимосвязи, обеспечивающие стабильную работу и удобство использования системы.

Программный продукт построен по модульному принципу, где каждый модуль отвечает за выполнение определённого набора операций, характерных для файловой системы. Такой подход облегчает сопровождение, расширение функционала и позволяет поэтапно развивать систему. Среди основных модулей можно выделить инициализацию и завершение работы, загрузку и сохранение JSON-файла, обработку операций с файлами и каталогами, а также вспомогательные функции логирования и обработки сигналов.

В основе системы лежит единый JSON-файл, выступающий в роли базы данных, который хранит иерархическую структуру каталогов и файлов. Каталоги представлены в виде JSON-объектов, где ключами являются имена файлов и подкаталогов, а значениями -

вложенные объекты или примитивные типы данных, соответствующие файлам. Такая модель обеспечивает естественное и прозрачное отображение файловой системы, позволяя использовать стандартные операции навигации и изменения данных.

Взаимодействие с пользователем и операционной системой осуществляется через стандартные системные вызовы, реализованные в виде обработчиков FUSE. Это обеспечивает полную совместимость с существующими инструментами и утилитами, а также позволяет пользователю работать с JSON FS без необходимости изучать новые интерфейсы или команды. Каждая операция сопровождается логированием, что повышает прозрачность и удобство отладки.

В структуре программного продукта предусмотрены механизмы, обеспечивающие целостность и безопасность данных. Среди них - фильтрация скрытых файлов и каталогов (ключи JSON, начинающиеся с точки, игнорируются), блокировка JSON-файла при операциях записи для предотвращения конфликтов, а также автоматическое создание родительских каталогов при необходимости. Обработка сигналов гарантирует корректное сохранение данных при завершении работы, что существенно повышает надёжность системы.

Архитектура программы учитывает особенности работы с файлами и каталогами, включая определение атрибутов (тип, права доступа, размер), чтение и запись содержимого, а также поддержку операций переименования и обрезки файлов. Все эти функции реализованы с учётом специфики JSON-формата и обеспечивают удобство и гибкость работы с данными.

В совокупности описанные компоненты и механизмы формируют целостный и сбалансированный программный продукт, сочетающий простоту использования, надёжность и функциональность. Структура программы обеспечивает эффективное управление данными, прозрачность операций и высокую степень совместимости с операционной системой и пользовательскими инструментами.

Таким образом подводя итог, файловая система JSON FS является современным, модульным и эргономичным решением для работы с JSON-данными в формате файловой системы. Её структура обеспечивает удобство сопровождения, расширяемость и надёжность, что делает продукт перспективным для практического применения и дальнейшего развития.

2.2 Защита от несанкционированного доступа

В разработанной файловой системе JSON FS защита от несанкционированного доступа реализована на нескольких уровнях, обеспечивая контроль прав доступа и предотвращение некорректного или вредоносного взаимодействия с данными.

2.2.1 Аутентификация и авторизация

Данная файловая система не содержит встроенного механизма аутентификации пользователей, так как взаимодействие с ней происходит через стандартный интерфейс операционной системы и её механизмы безопасности. Аутентификация пользователей и контроль прав доступа возлагаются на операционную систему, под управлением которой монтируется файловая система.

При этом JSON FS наследует права доступа и идентификаторы пользователей (UID и GID) из ОС, что позволяет использовать стандартные механизмы управления доступом:

- При выполнении операций с файлами и каталогами в методах, таких как `jsonfs_getattr`, `jsonfs_open`, `jsonfs_write` и других, устанавливаются права доступа, аналогичные стандартным UNIX-права (например, 0755 для каталогов и 0644 для файлов).
- В структуре атрибутов файлов (`struct stat`) указываются UID и GID текущего пользователя, что позволяет ОС корректно проверять права доступа при попытках чтения, записи или выполнения операций с файлами.

Таким образом, доступ к JSON FS контролируется через стандартные механизмы ОС, и только пользователи с соответствующими правами могут выполнять операции с файлами и каталогами.

2.2.2 Ограничение доступа к JSON-файлу

Физический JSON-файл, в котором хранится вся структура файловой системы, должен иметь соответствующие права доступа на уровне файловой системы ОС. В частности:

- Файл доступен только для чтения и записи пользователям и процессам, которым разрешён доступ к монтируемой файловой системе.
- При инициализации файловой системы (функция `jsonfs_init`) проверяется доступность JSON-файла с правами на чтение и запись. В случае отсутствия необходимых прав монтирование не происходит, а в системный журнал записывается ошибка.

2.2.3 Блокировка файла при записи

Для предотвращения одновременного изменения JSON-файла несколькими процессами реализована файловая блокировка:

- При открытии файла с флагами записи (`O_WRONLY` или `O_RDWR`) вызывается системный вызов `flock` с эксклюзивной блокировкой (`LOCK_EX`).

– Это гарантирует, что в момент записи JSON-файл не будет изменяться другими процессами, что предотвращает повреждение данных и обеспечивает целостность информации.

– После завершения записи и закрытия файла блокировка снимается (LOCK_UN).

2.2.4 Ограничение создания и работы с скрытыми файлами

В файловой системе реализована фильтрация скрытых файлов и каталогов - все элементы, имена которых начинаются с точки (.), игнорируются и не отображаются в списках каталогов. Кроме того, создание таких файлов запрещено (возвращается ошибка EPERM).

Это снижает риск случайного или преднамеренного создания скрытых объектов, которые могут использоваться для обхода контроля доступа или сокрытия данных.

2.2.5 Логирование операций

Все операции с файловой системой (создание, удаление, чтение, запись, переименование и прочие) логируются через системный журнал (syslog). Логирование включает информацию о типе операции и пути к файлу или каталогу.

Логирование позволяет отслеживать действия пользователей и выявлять подозрительную активность, что является важной частью системы защиты.

2.2.6 Обработка сигналов и безопасное завершение работы

При получении сигналов прерывания (SIGINT, SIGTERM) реализован обработчик, который гарантирует сохранение всех изменений в JSON-файл и корректное освобождение ресурсов. Это предотвращает потерю данных и обеспечивает стабильность работы системы.

2.2.7 Подробное описание реализации защиты на примере кода

3.2.7.1 Контроль прав доступа через UID и GID

Файловая система JSON FS работает в пространстве пользователя и использует возможности операционной системы для контроля доступа к файлам и каталогам. В частности, при выполнении операции получения атрибутов файла или каталога (функция jsonfs_getattr) устанавливаются права доступа и идентификаторы пользователя и группы, что позволяет ОС осуществлять проверку прав доступа.

Фрагмент кода 1 – Функции jsonfs_getattr

```
int jsonfs_getattr(const char *path, struct stat *stbuf) {
    memset(stbuf, 0, sizeof(struct stat));
    if(strncmp(path, "/", 1) == 0) {
        stbuf->st_mode = S_IFDIR | 0755; // Каталог с правами rwxr-xr-x
        stbuf->st_nlink = 2;
    } else {
        // Определение типа объекта (каталог или файл)
        if(json_is_object(current)) {
            stbuf->st_mode = S_IFDIR | 0755;
        } else {
```



```

        stbuf->st_mode = S_IFREG | 0644; // Файл с правами rw-r--r--
    }
}
stbuf->st_uid = getuid(); // Устанавливаем владельца - текущий пользователь
stbuf->st_gid = getgid(); // Устанавливаем группу - текущая группа
stbuf->st_atime = stbuf->st_mtime = stbuf->st_ctime = time(NULL);
return 0;
}

```

В данном коде для каждого файла и каталога устанавливаются стандартные UNIX-права доступа: каталоги получают права 0755, а файлы - 0644. Кроме того, в атрибуты записываются идентификаторы пользователя (UID) и группы (GID), соответствующие текущему процессу. Это позволяет операционной системе контролировать доступ к элементам файловой системы в соответствии с установленными правами.

3.2.7.2 Проверка доступа к JSON-файлу при инициализации

Перед монтированием файловой системы осуществляется проверка прав доступа к исходному JSON-файлу, в котором хранится структура файловой системы. Эта проверка реализована в функции `jsonfs_init` и предотвращает работу с файлом, если у процесса нет прав на чтение и запись.

Фрагмент кода 2 – Функции `jsonfs_init`

```

void* jsonfs_init(struct fuse_conn_info *conn) {
    if(access(json_file, R_OK | W_OK) != 0) {
        syslog(LOG_ERR, "Доступ к JSON-файлу запрещён: %s", strerror(errno));
        return NULL;
    }
    load_json();
    if(!root) {
        syslog(LOG_ERR, "Ошибка инициализации корня JSON");
        return NULL;
    }
    return NULL;
}

```

Здесь системный вызов `access` проверяет наличие прав на чтение и запись файла. В случае отсутствия необходимых прав монтирование файловой системы прерывается, а ошибка фиксируется в системном журнале. Таким образом, обеспечивается защита на уровне файловой системы операционной системы.

Здесь системный вызов `access` проверяет наличие прав на чтение и запись файла. В случае отсутствия необходимых прав монтирование файловой системы прерывается, а ошибка фиксируется в системном журнале. Таким образом, обеспечивается защита на уровне файловой системы операционной системы.

3.2.7.3 Файловая блокировка при записи

Для предотвращения одновременного изменения JSON-файла несколькими процессами реализован механизм блокировки. При открытии файла для записи вызывается

функция flock с эксклюзивной блокировкой, которая удерживается до закрытия файла.

Фрагмент кода 3 – Функции jsonfs_open

```
int jsonfs_open(const char *path, struct fuse_file_info *fi) {
    json_t *file = get_json_by_path(path + 1);
    if(!file) return -ENOENT;
    if(json_is_object(file)) return -EISDIR;
    if(fi->flags & (O_WRONLY | O_RDWR)) {
        int fd = open(json_file, O_RDWR);
        if(fd < 0) {
            syslog(LOG_ERR, "Не удалось открыть JSON-файл: %s",
strerror(errno));
            return -errno;
        }
        if(flock(fd, LOCK_EX) == -1) {
            close(fd);
            syslog(LOG_ERR, "Не удалось установить блокировку: %s",
strerror(errno));
            return -errno;
        }
        fi->fh = (uint64_t)fd;
    }
    return 0;
}
```

И соответствующая разблокировка в функции jsonfs_release:

Фрагмент кода 4 – Функции jsonfs_release

```
int jsonfs_release(const char *path, struct fuse_file_info *fi) {
    if(fi->fh) {
        int fd = (int)fi->fh;
        save_json();
        flock(fd, LOCK_UN);
        close(fd);
    }
    return 0;
}
```

Данный механизм гарантирует, что в момент записи JSON-файл не будет изменён другим процессом, что исключает риск повреждения данных и обеспечивает целостность информации.

3.2.7.4 Запрет создания скрытых файлов

В целях безопасности реализован запрет на создание файлов и каталогов, имена которых начинаются с точки, что соответствует концепции скрытых файлов в UNIX-системах. Это предотвращает возможность сокрытия файлов и каталогов внутри JSON FS.

Фрагмент кода 5 – Функции jsonfs_create

```
int jsonfs_create(const char *path, mode_t mode, struct fuse_file_info *fi) {
    char *path_copy = strdup(path);
    char *name = basename(path_copy);
    if(name[0] == '.') {
        free(path_copy);
        return -EPERM; // Запрет создания скрытых файлов
    }
}
```

```

        // ...
        free(path_copy);
        return 0;
    }

```

Аналогично, при чтении содержимого каталога скрытые файлы исключаются из списка в функции `jsonfs_readdir`.

Фрагмент кода 6 – Функции `jsonfs_readdir`

```

int jsonfs_readdir(const char *path, void *buf, fuse_fill_dir_t filler, off_t
offset, struct fuse_file_info *fi) {
    log_operation("READDIR", path);
    filler(buf, ".", NULL, 0);
    filler(buf, "..", NULL, 0);
    json_t *dir = get_json_by_path(path + 1);
    if(!dir || !json_is_object(dir)) return -ENOENT;
    const char *key;
    json_t *value;
    json_object_foreach(dir, key, value) {
        if(key[0] != '.') { // Исключение скрытых файлов
            if(filler(buf, key, NULL, 0) != 0) break;
        }
    }
    return 0;
}

```

Таким образом в данной реализации, скрытые файлы и каталоги не отображаются пользователю и не могут быть созданы, что повышает уровень безопасности и предотвращает злоупотребления.

3.2.7.5 Логирование операций

Для контроля и аудита всех операций с файловой системой реализовано логирование через системный журнал (`syslog`). Каждое действие пользователя, связанное с файловой системой, фиксируется с указанием типа операции и пути к файлу или каталогу.

Фрагмент кода 7 – Функция логирования `log_operation`

```

void log_operation(const char *op, const char *path) {
    openlog("jsonfs", LOG_PID | LOG_CONS, LOG_USER);
    syslog(LOG_INFO, "[%s] %s", op, path);
    closelog();
}

```

Вызов логирования присутствует во всех основных операциях. В качестве примера можно привести фрагмент функции `jsonfs_read`.

Фрагмент кода 8 – Функции `jsonfs_read`

```

int jsonfs_read(const char *path, char *buf, size_t size, off_t offset, struct
fuse_file_info *fi) {
    log_operation("READ", path); // Логирование
    // ...
}

```

Логирование позволяет отслеживать действия пользователей, выявлять попытки несанкционированного доступа и анализировать поведение системы.

3.2.7.6 Обработка сигналов и безопасное завершение работы

Для предотвращения потери данных при аварийном завершении работы системы реализован обработчик сигналов, который сохраняет текущее состояние JSON-файла и корректно завершает работу.

Фрагмент кода 9 – Обработчик сигналов sig_handler

```
void sig_handler(int signo) {  
    if(root) {  
        save_json();  
        sync();  
    }  
    exit(0);  
}
```

Сам обработчик регистрируется на сигналы прерывания и завершения.

Фрагмент кода 10 – Обработчики сигналов SIGINT и SIGTERM

```
signal(SIGINT, sig_handler);  
signal(SIGTERM, sig_handler);
```

SIGINT - это сигнал прерывания, который обычно посылается при нажатии Ctrl-C в терминале. Он служит для прерывания выполнения процесса.

SIGTERM - это сигнал завершения, который используется для корректного завершения процесса. Его можно послать процессу извне, и он позволяет процессу выполнить очистку перед выходом

Таким образом, это гарантирует, что при получении сигналов завершения работы все изменения будут сохранены, а ресурсы освобождены корректно.

2.2.8 Вывод защиты от несанкционированного доступа

Подводя под итог защиты от несанкционированного доступа, разработанная файловая система JSON FS обладает комплексной системой защиты от несанкционированного доступа, что обеспечивает высокий уровень безопасности и целостности данных при её эксплуатации. Использование стандартных механизмов операционной системы для контроля прав доступа позволяет эффективно ограничивать возможности пользователей в соответствии с их полномочиями. В частности, наследование идентификаторов пользователя и группы, а также установка соответствующих прав доступа к файлам и каталогам создают надёжный базис для контроля доступа на уровне ОС.

Кроме того, предусмотрена проверка прав доступа к исходному JSON-файлу при инициализации файловой системы. Это предотвращает возможность монтирования и работы с файлом в случае отсутствия необходимых прав, что снижает риск несанкционированного доступа и повреждения данных. В случае возникновения проблем с доступом система информирует пользователя через системный журнал и стандартный вывод ошибок,

обеспечивая прозрачность и оперативность реагирования.

Для предотвращения конфликтов и повреждений данных при одновременной работе нескольких процессов реализован механизм файловой блокировки на время записи. Эксклюзивная блокировка JSON-файла гарантирует, что в момент изменения данных доступ к файлу имеет только один процесс, что значительно повышает надёжность и устойчивость системы. В случае невозможности установить блокировку операция записи прерывается с соответствующим уведомлением, что позволяет своевременно выявлять и устранять потенциальные проблемы.

Дополнительным уровнем защиты служит запрет на создание скрытых файлов и каталогов, имена которых начинаются с точки. Это ограничение исключает возможность сокрытия вредоносных или нежелательных объектов внутри файловой системы, повышая безопасность и упрощая управление данными. При попытке создания таких объектов система возвращает ошибку, информируя пользователя о недопустимости операции.

Логирование всех операций с файловой системой через системный журнал обеспечивает возможность аудита и мониторинга действий пользователей. Это важный инструмент для выявления подозрительной активности и предотвращения несанкционированных вмешательств. Информативные сообщения об ошибках и событиях позволяют быстро реагировать на инциденты и обеспечивают прозрачность работы системы.

Обработка сигналов прерывания и завершения работы гарантирует сохранение всех изменений и корректное освобождение ресурсов, что снижает риск потери данных при аварийных ситуациях. Такой подход способствует повышению общей устойчивости и безопасности программного продукта.

В совокупности реализованные механизмы защиты обеспечивают надёжную и эффективную систему контроля доступа, способную предотвращать несанкционированные действия и сохранять целостность данных. Это делает файловую систему JSON FS безопасным и надёжным инструментом для работы с JSON-данными в различных условиях эксплуатации.

2.3 Демонстрация функциональных возможностей

В данном разделе будет подробно описывается выполнение основных функций файловой системы JSON FS, разработанной для удобного и эффективного управления данными в формате JSON через стандартный файловый интерфейс ОС ядра Linux. Каждая функция сопровождается подробным описанием последовательности действий, входными данными, ожидаемыми результатами, а также анализом корректности выполнения. Для наглядности приведены примеры команд, результаты их выполнения и рекомендации по

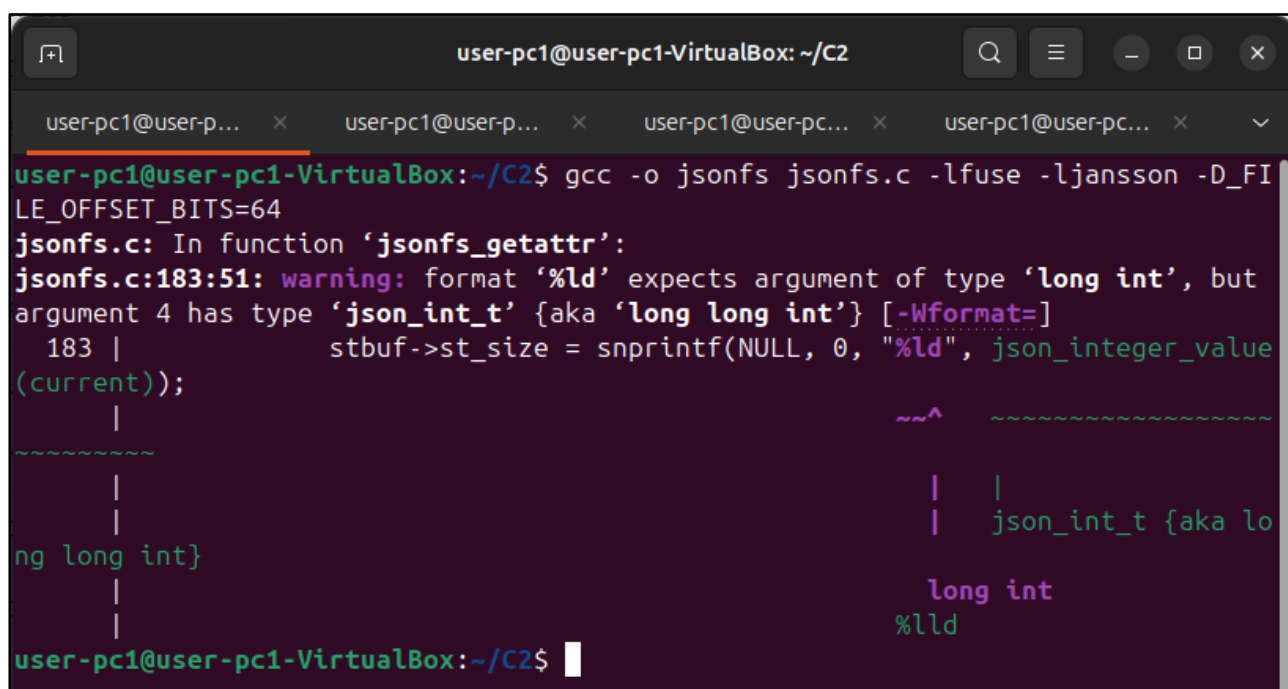
использованию.

2.3.1 Сборка и компиляция виртуальной файловой системы JSON FS

Для сборки и компиляции программы JSON FS (виртуальной файловой системы) необходимо выполнить следующие основные действия:

- Открыть терминал и перейти в каталог с исходным кодом.
- Выполнить команду для компиляции: `gcc -o «название программы» «исходный файл для компиляции» -lfuse -ljansson -D_FILE_OFFSET_BITS=64`.

Пример: `gcc -o jsonfs jsonfs.c -lfuse -ljansson -D_FILE_OFFSET_BITS=64` (см. скриншот 1).



```
user-pc1@user-pc1-VirtualBox: ~/C2
user-pc1@user-pc1-VirtualBox:~/C2$ gcc -o jsonfs jsonfs.c -lfuse -ljansson -D_FILE_OFFSET_BITS=64
jsonfs.c: In function 'jsonfs_getattr':
jsonfs.c:183:51: warning: format '%ld' expects argument of type 'long int', but
183 |         stbuf->st_size = snprintf(NULL, 0, "%ld", json_integer_value
    |                                     ~~~~~^~~~~~
    |                                     |         |
    |                                     |         json_int_t {aka long long int}
    |                                     long int
    |                                     %lld
user-pc1@user-pc1-VirtualBox:~/C2$
```

Рисунок 1 – Процесс компиляции программы JSON FS



Рисунок 2 – Полученный исполняемый файл программы JSON FS

Входные данные:

- Исходный код программы: файл jsonfs.c (или другой, содержащий реализацию файловой системы).
- Установленные библиотеки: FUSE (например, пакет libfuse-dev) и Jansson (например, пакет libjansson-dev)
- Компилятор GCC (или совместимый).

Ожидаемый результат:

- Успешная компиляция исходного кода без ошибок.
- Получение исполняемого файла (см. скриншот 2), например, jsonfs, в текущем каталоге.

Результаты и выводы:

- Программа успешно собрана в каталоге с исходным кодом и появился исполняемый файл, а также готова к дальнейшему использованию.
- В терминале отсутствуют сообщения об ошибках компиляции (возможны только предупреждения).
- Все необходимые зависимости (FUSE, Jansson) корректно установлены.
- На данном этапе можно переходить к монтированию виртуальной файловой системы и тестированию её функциональности.

2.3.2 Монтирование JSON-файла как виртуальной файловой системы

Для монтирования JSON-файла как виртуальной файловой системы необходимо выполнить следующие основные действия:

- Запуск исполняемого файла JSON FS с указанием пути к исходному JSON-файлу и точки монтирования в файловой системе Linux.
- Проверка успешного монтирования и доступности структуры JSON как каталогов и файлов.

Пример команды: `./jsonfs /mnt/jsonfs test.json` (см. скриншот 3).

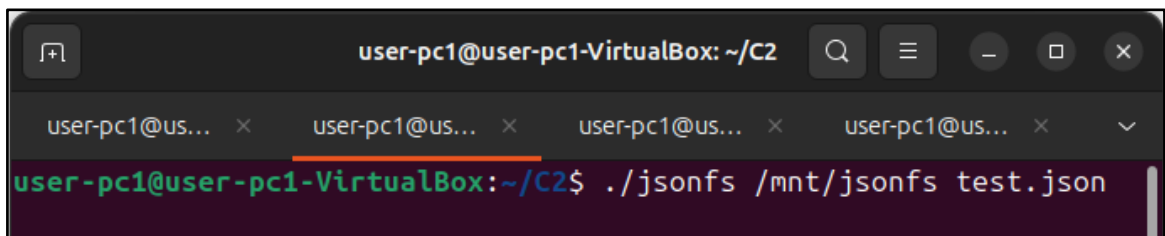


Рисунок 3 – Ввод команды для монтирования файловой системы JSON-файла программой JSON FS

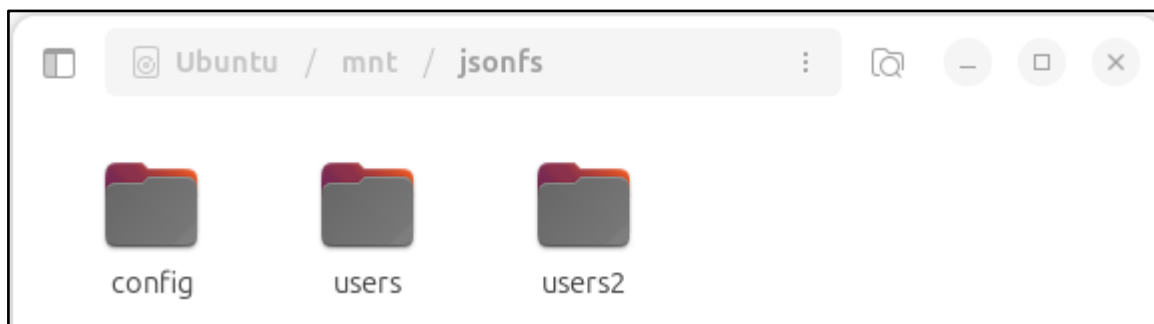


Рисунок 4 – Результат монтирования файловой системы JSON-файла программой JSON FS

Входные данные:

- JSON-файл test.json, содержащий иерархическую структуру данных с несколькими уровнями вложенности.
- Пустая директория /mnt/jsonfs, предназначенная для монтирования.

Ожидаемые результаты:

JSON-файл успешно монтируется как виртуальная файловая система.

- В каталоге /mnt/jsonfs появляется структура каталогов и файлов, соответствующая объектам и параметрам JSON (см. скриншот 4).
- Пользователь может просматривать содержимое с помощью стандартных команды ls.

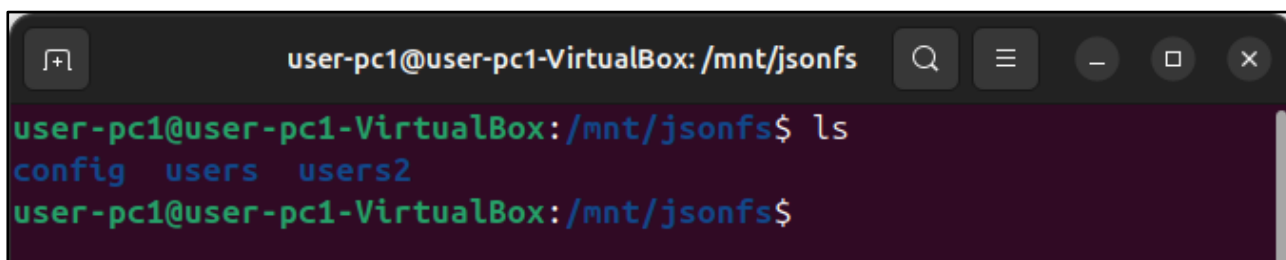


Рисунок 5 – Результат просмотра содержимого JSON-файла командой ls программой JSON FS

Результаты и выводы:

После выполнения команды монтирования структура JSON становится доступной для навигации и управления через файловую систему. Проверка с помощью ls -R /mnt/jsonfs демонстрирует полное соответствие отображаемой структуры исходному JSON-файлу. Это подтверждает корректность реализации механизма монтирования и интеграции с операционной системой.

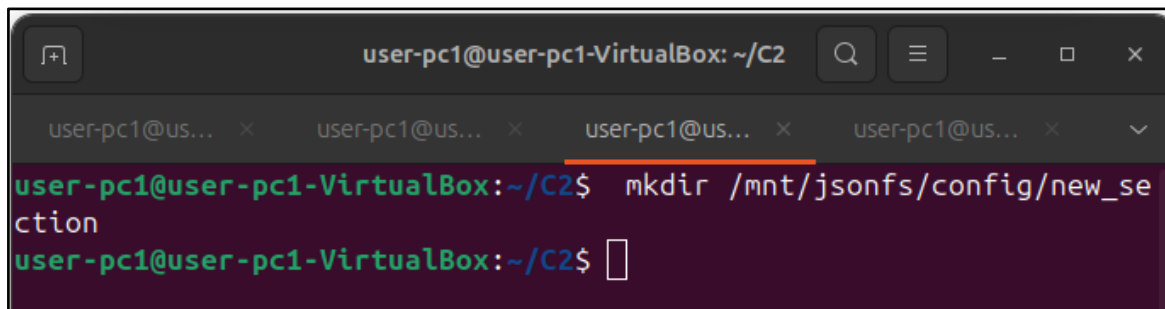
2.3.3 Создание нового раздела (каталога) в JSON FS

Основные действия:

- Использование команды mkdir для создания нового раздела (объекта JSON) в виртуальной файловой системе.

- Автоматическое обновление исходного JSON-файла с добавлением нового объекта.

Пример команды: `mkdir /mnt/jsonfs/config/new_section` (см. скриншот 6).



```
user-pc1@user-pc1-VirtualBox: ~/C2
user-pc1@us... x user-pc1@us... x user-pc1@us... x user-pc1@us... x
user-pc1@user-pc1-VirtualBox:~/C2$ mkdir /mnt/jsonfs/config/new_section
user-pc1@user-pc1-VirtualBox:~/C2$
```

Рисунок 6 – Создание нового раздела (каталога) в JSON-файле программой JSON FS

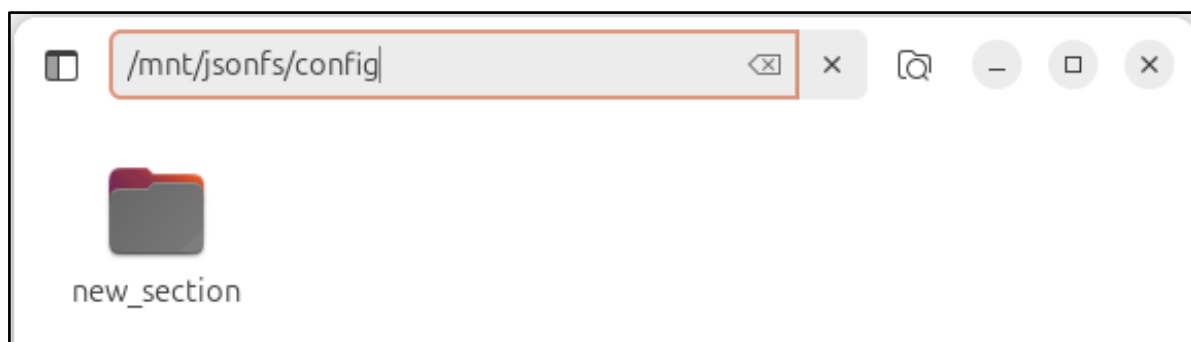


Рисунок 7 – Результат создания нового раздела (каталога) в JSON-файле программой JSON FS

Входные данные:

- Путь к создаваемому каталогу `/mnt/jsonfs/config/new_section`.
- Исходный JSON-файл, в котором отсутствует раздел `new_section`.

Ожидаемые результаты:

- В виртуальной файловой системе появляется новый каталог `new_section` внутри `config` (см. скриншот 7).
- В JSON-файле добавляется соответствующий объект с пустой структурой.
- Пользователь может создавать внутри нового раздела параметры и подкаталоги.

Результаты и выводы:

Создание раздела прошло успешно, что подтверждается выводом команды `ls /mnt/jsonfs/config` - новый каталог отображается корректно. Анализ JSON-файла показывает наличие нового объекта, соответствующего созданному каталогу. Это свидетельствует о правильной реализации операции создания объектов и обновления данных.

2.3.4 Создание нового файла (параметра) в JSON FS

Основные действия:

- Использование команды `cat >` для создания нового файла (объекта JSON) в виртуальной файловой системе.
- Автоматическое обновление исходного JSON-файла с добавлением нового объекта.

Пример команды: `cat > /mnt/jsonfs/config/timeout` (см. скриншот 8).

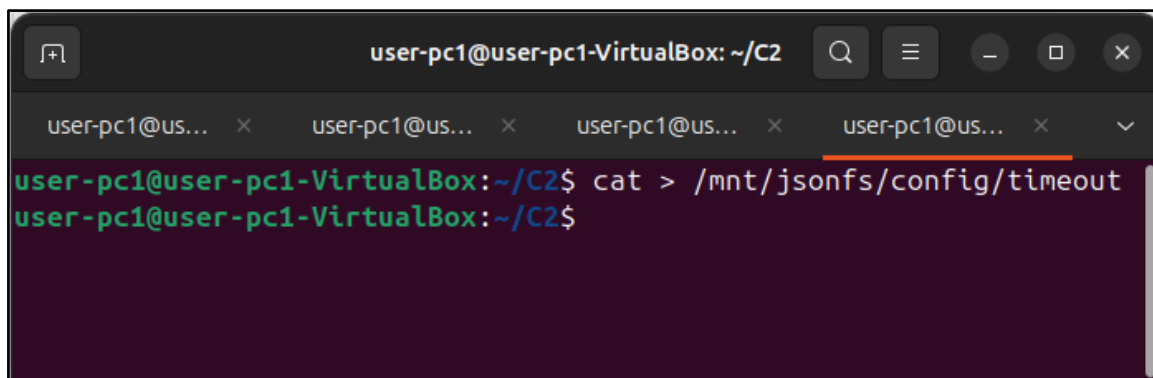


Рисунок 8 – Создание нового файла (параметра) в JSON-файле программой JSON FS

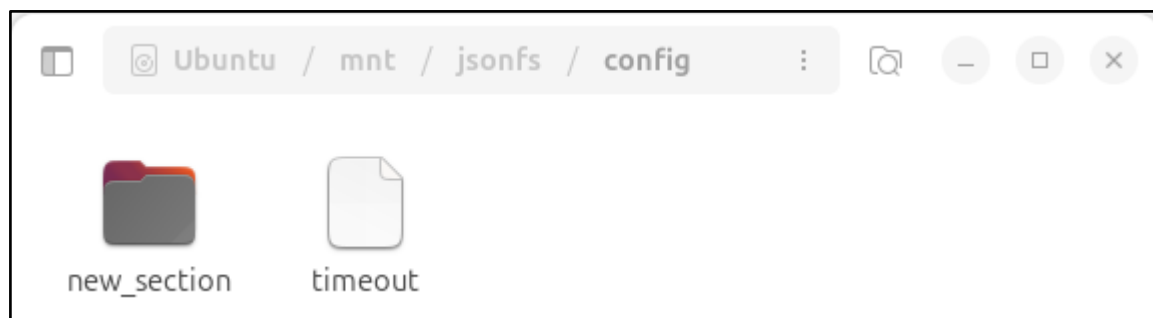


Рисунок 9 – Результат создания нового файла (параметра) в JSON-файле программой JSON FS

Входные данные:

- Путь к создаваемому файлу `/mnt/jsonfs/config/`.
- Исходный JSON-файл, в котором отсутствует файл (параметр) `timeout`.

Ожидаемые результаты:

- В виртуальной файловой системе появляется новый файл (параметр) `timeout` внутри каталога `config` (см. скриншот 9).
- В JSON-файле добавляется соответствующий объект с пустой структурой.
- Пользователь может создавать внутри нового раздела параметры и подкаталоги.

Результаты и выводы:

Создание файла прошло успешно, что подтверждается выводом команды `ls /mnt/jsonfs/config` - новый файл отображается корректно. Анализ JSON-файла показывает наличие нового объекта, соответствующего созданному файлу. Это свидетельствует о

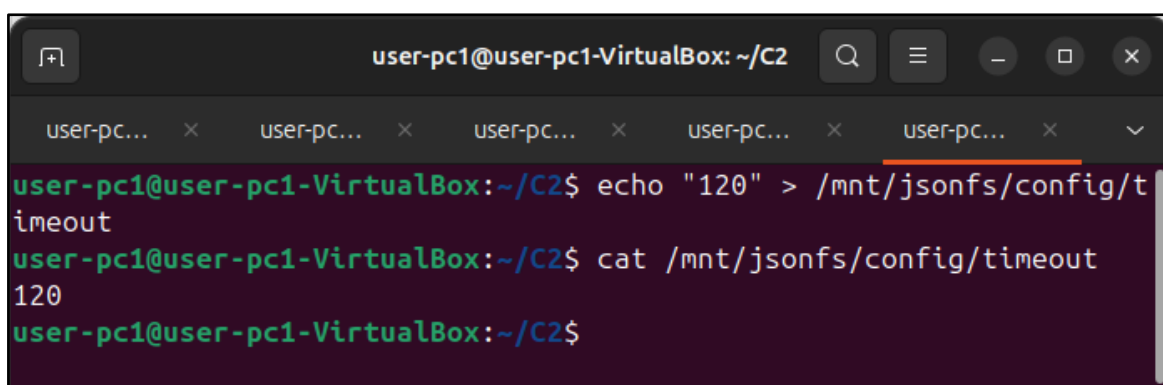
правильной реализации операции создания объектов и обновления данных.

2.3.5 Изменение значения параметра JSON

Основные действия:

- Использование команды `echo` с перенаправлением для записи нового значения в файл параметра.
- Автоматическое преобразование введённого текста в соответствующий тип JSON.
- Обеспечение сохранности и целостности JSON-файла.

Пример команды: `echo "120" > /mnt/jsonfs/config/timeout` (см. скриншот 10).



```
user-pc1@user-pc1-VirtualBox: ~/C2
user-pc1@user-pc1-VirtualBox:~/C2$ echo "120" > /mnt/jsonfs/config/timeout
user-pc1@user-pc1-VirtualBox:~/C2$ cat /mnt/jsonfs/config/timeout
120
user-pc1@user-pc1-VirtualBox:~/C2$
```

Рисунок 10 – Чтение и изменение значения параметра в JSON-файле программой JSON FS

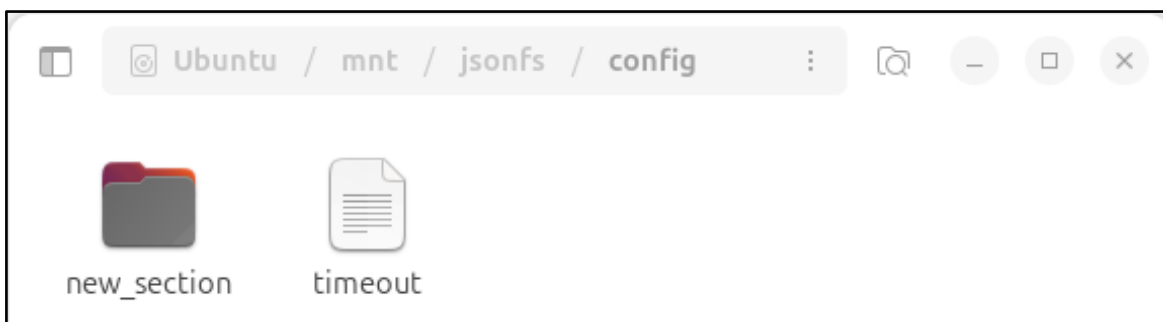


Рисунок 11 – Результат изменения значения параметра в JSON-файле программой JSON FS

Входные данные:

- Новый параметр `timeout` со значением 120.
- Путь к файлу параметра.

Ожидаемые результаты:

- Обновление значения параметра в JSON-файле (см. скриншот 11).
- При последующем чтении отображается новое значение.

Результаты и выводы:

После выполнения команды новое значение успешно сохраняется. Проверка с помощью `cat` показывает обновлённое значение 120. Анализ JSON-файла подтверждает

корректное обновление данных без повреждений. Это свидетельствует о надёжности механизма записи и преобразования данных.

2.3.6 Чтение значения параметра JSON

Основные действия:

- Использование команды `cat` для просмотра значения параметра, представленного в виде файла.
- Поддержка различных типов данных: строки, числа, булевы значения.

Пример команды: `cat /mnt/jsonfs/config/timeout` (см. скриншот 12)

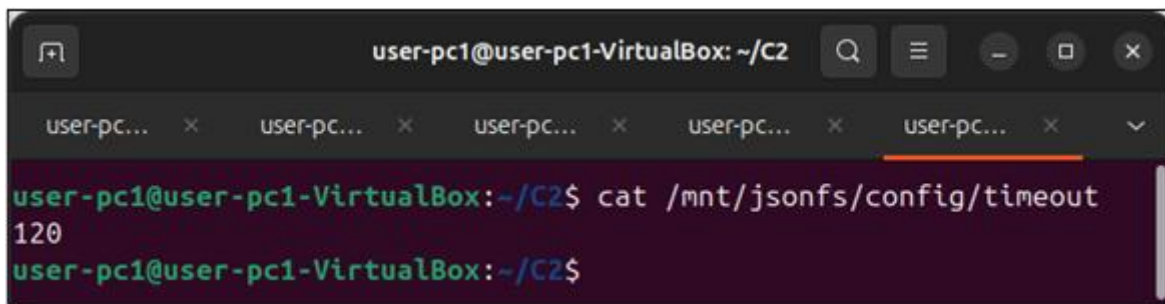


Рисунок 12 – Чтение и изменение значения параметра в JSON-файле программой JSON FS

Входные данные:

- Путь к файлу параметра `timeout`.
- Значение параметра в JSON-файле, например, число 120.

Ожидаемые результаты:

- Отображение значения параметра в текстовом виде.
- Корректное преобразование типов данных в читаемый формат.

Результаты и выводы:

Команда выводит значение 120, совпадающее с данными в JSON-файле. Аналогично корректно отображаются строки и булевы значения (`true/false`). Это подтверждает корректность реализации чтения параметров и преобразования типов.

2.3.7 Удаление разделов и параметров JSON

Основные действия:

- Использование команды `rm` для удаления файлов (параметров) и каталогов (разделов).
- Обработка ошибок при попытках удаления несуществующих или защищённых элементов.

Пример команды удаления файлов (параметров): `rm /mnt/jsonfs/config/timeout` (см. скриншот 13).

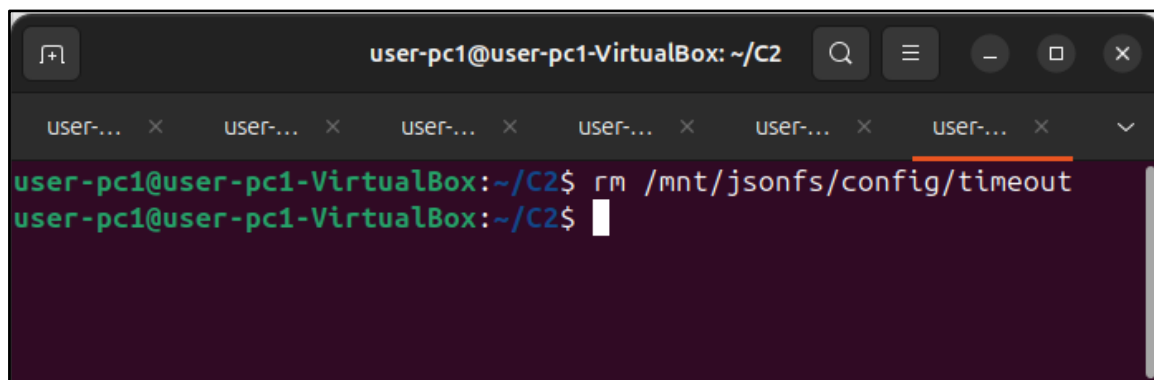


Рисунок 13 – Удаление параметра в JSON FS в JSON-файле программой JSON FS



Рисунок 14 – Результат удаления параметра в JSON FS в JSON-файле программой JSON FS

Пример команды удаления каталогов (1-й вариант): **rm -r /mnt/jsonfs/config/new_section** (см. скриншот 15).

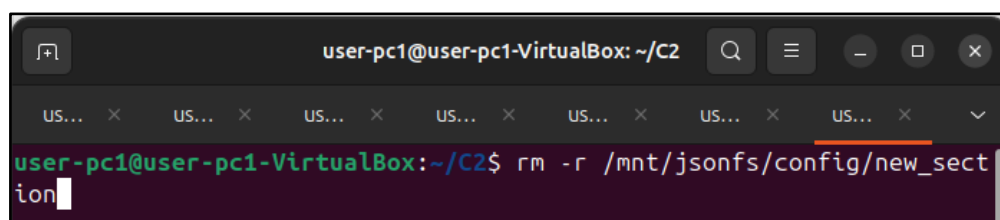


Рисунок 15 – Команда удаления раздела в JSON-файле программой JSON FS



Рисунок 16 – Каталог «new_section» до его удаления в JSON-файле программой JSON FS

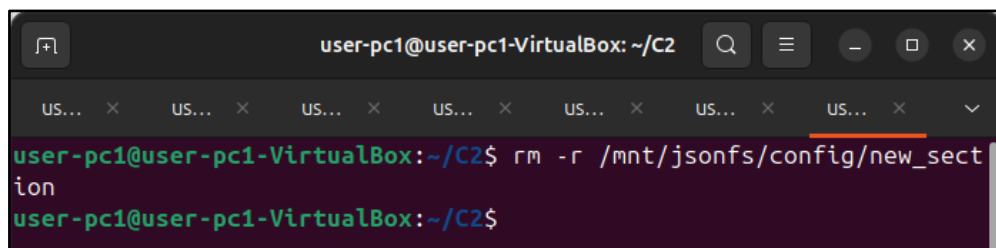


Рисунок 17 – Используемая команда удаления раздела в JSON-файле программой JSON FS

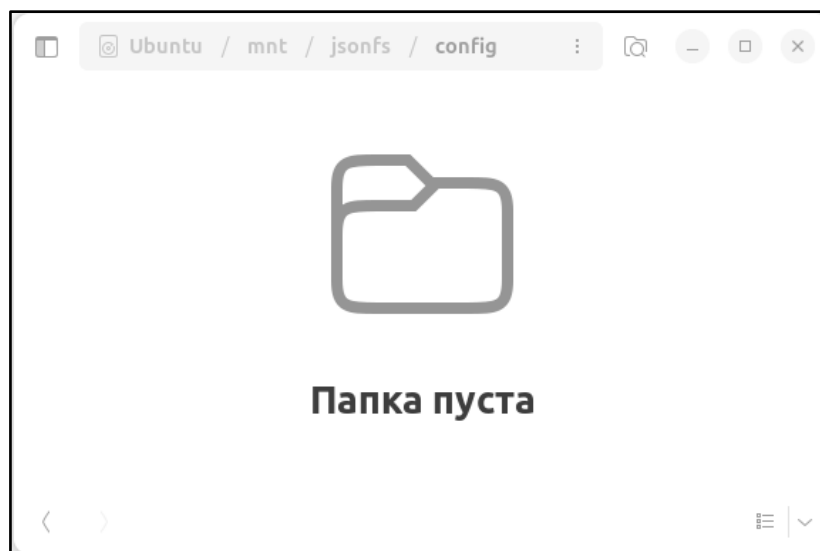


Рисунок 18 – Результат удаления раздела командой в JSON-файле программой JSON FS

Пример команды удаления каталогов (2-й вариант): **rmdir** /mnt/jsonfs/config/old_section (см. скриншот 19).

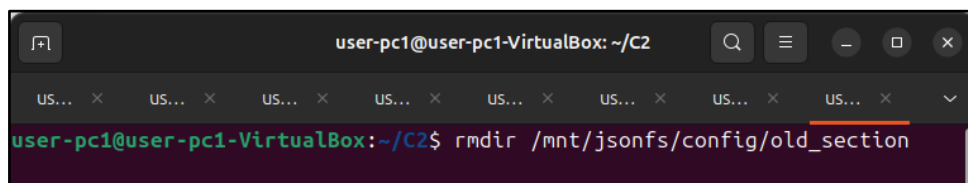


Рисунок 19 – Команда удаления раздела в JSON-файле программой JSON FS (2 способ)

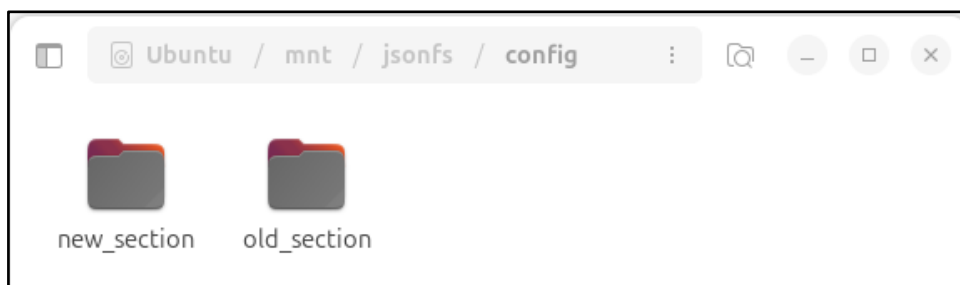


Рисунок 20 – Каталог «old_section» до его удаления в JSON-файле программой JSON FS

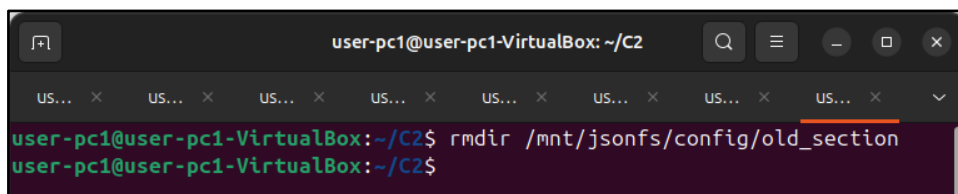


Рисунок 21 – Используемая команда удаления раздела в JSON-файле программой JSON FS
(2 способ)



Рисунок 22 – Результат удаления раздела командой в JSON-файле программой JSON FS

Входные данные:

- Путь к удаляемому параметру и/или разделу.

Ожидаемые результаты:

- Удаление соответствующих элементов из JSON-структуры и виртуальной файловой системы. Результат удаления файла см. скриншот 14. Результат удаления каталога см. скриншот 18 (1-го варианта) и см. скриншот 22 (2-го варианта).
- Вывод ошибок при попытках удаления отсутствующих объектов.

Результаты и выводы:

Удаление прошло успешно, что подтверждается отсутствием удалённых элементов при последующем просмотре. При попытке удалить несуществующий файл система корректно выводит сообщение об ошибке. Это демонстрирует устойчивость и корректность обработки операций удаления.

2.3.8 Переименование и перемещение файлов и каталогов

Основные действия:

- Использование команды `mv` для переименования и перемещения объектов внутри файловой системы.
- Обновление структуры JSON с учётом изменений.

Пример команды переименования файла: `mv /mnt/jsonfs/config/timeout /mnt/jsonfs/config/connection_timeout` (см. скриншот 23).

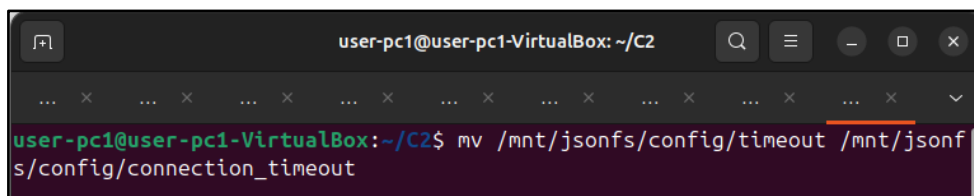


Рисунок 23 – Команда переименования файла в JSON-файле программой JSON FS

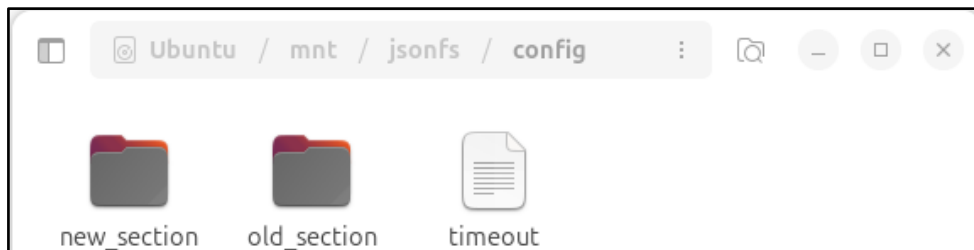


Рисунок 24 – Файл «timeout» до его переименования в JSON-файле программой JSON FS

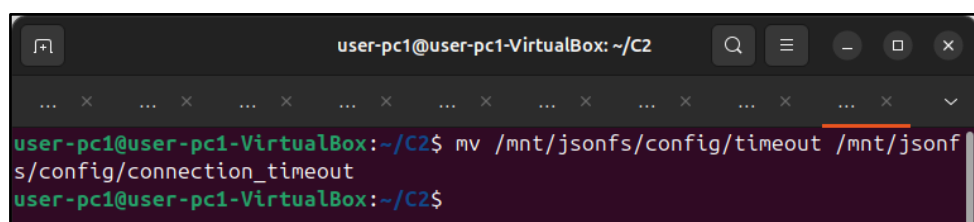


Рисунок 25 – Используемая команда переименования файла в JSON-файле программой JSON FS

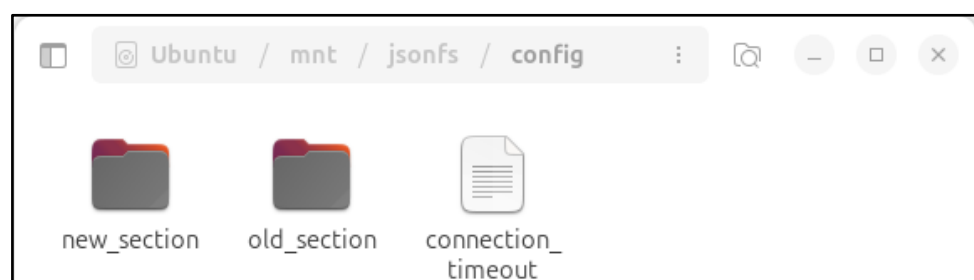


Рисунок 26 – Результат переименования файла «timeout» в «connection_timeout» в JSON-файле программой JSON FS

Пример команды переименования файла: **mv /mnt/jsonfs/old_section /mnt/jsonfs/new_section/old_section** (см. скриншот 27).

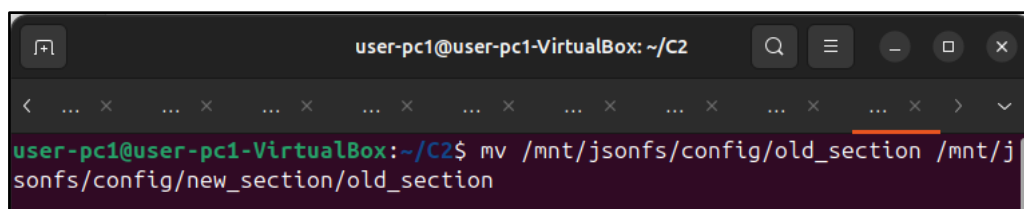


Рисунок 27 – Команда перемещения каталога в JSON-файле программой JSON FS



Рисунок 28 – Каталог «old_section» до его перемещения в JSON-файле программой JSON FS

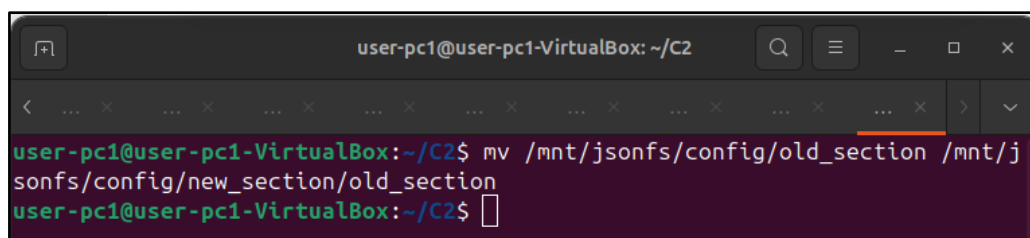


Рисунок 29 – Используемая команда перемещения каталога в JSON-файле программой JSON FS

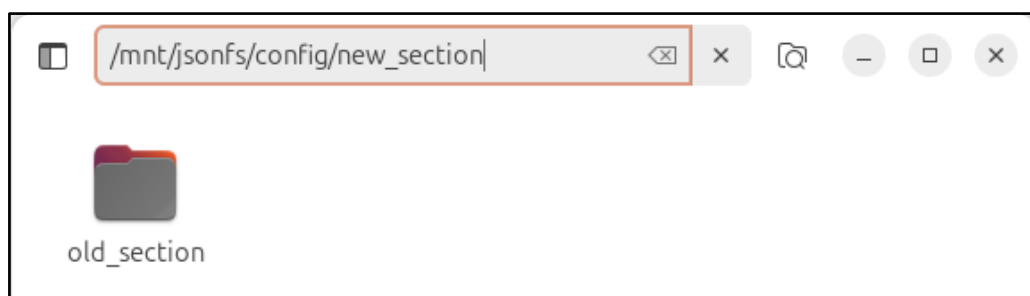


Рисунок 30 – Результат команды перемещения каталога «old_section» в «new_section» в JSON-файле программой JSON FS

Входные данные:

- Исходные и целевые пути объектов.

Ожидаемые результаты:

- Объекты успешно переименовываются (см. скриншот 26) и перемещаются (см. скриншот 30) в структуре JSON и файловой системе.
- Сохранение целостности данных.

Результаты и выводы:

Команды выполняются без ошибок, структура JSON обновляется корректно. Проверка с помощью ls и просмотра JSON-файла подтверждает успешное переименование и перемещение. Это свидетельствует о правильной реализации данной функциональности.

2.3.9 Обработка ошибок и сообщений системе

Основные действия:

- Отслеживание и вывод сообщений об ошибках (см. приложение Б) при выполнении операций.

- Информирование пользователя о причинах сбоев и рекомендациях.

Пример сообщений:

- Error: JSON file access denied - недостаточно прав для доступа к JSON-файлу.
- Error loading JSON file: syntax error - ошибка в структуре JSON.
- File lock failed - невозможность установить блокировку файла.
- Operation not permitted - запрещённая операция (например, создание скрытого файла).

Более подробно с возможными ошибками при работе с программой можете ознакомиться в таблице 2 – сообщения программы при её некорректном выполнении.

Входные данные:

- Ошибочные действия пользователя или системные сбои.

Ожидаемые результаты:

- Вывод информативных сообщений с описанием проблемы.
- Возможность предпринять корректирующие действия.

Результаты и выводы:

Сообщения отображаются своевременно и содержат полезную информацию для устранения проблем. Это повышает удобство эксплуатации и надёжность работы системы.

2.3.10 Завершение работы и размонтирование файловой системы

Основные действия:

- Выполнение команды размонтирования (см. приложение Н) с сохранением всех изменений.

Пример команды: **fusermount -u /mnt/jsonfs** или **sudo umount /mnt/jsonfs** (см. скриншот 31).

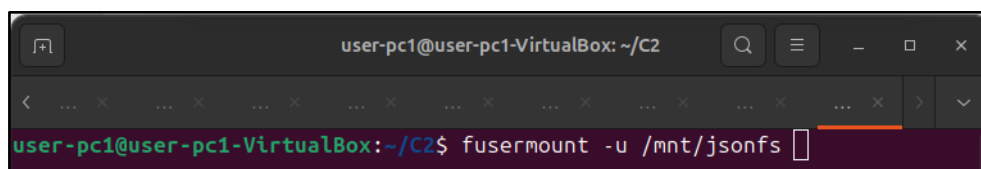


Рисунок 31 – Команда для завершения работы и размонтирования файловой системы JSON

FS

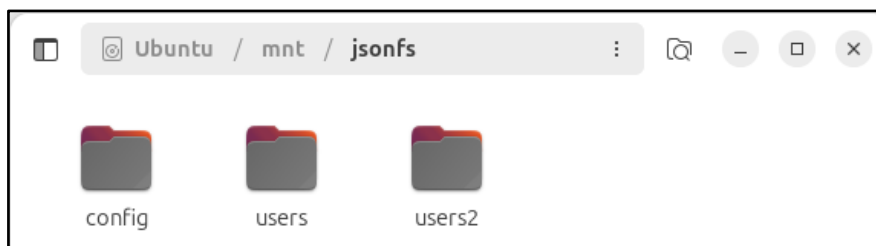


Рисунок 32 – Файловая система JSON FS до её завершения работы и размонтирования

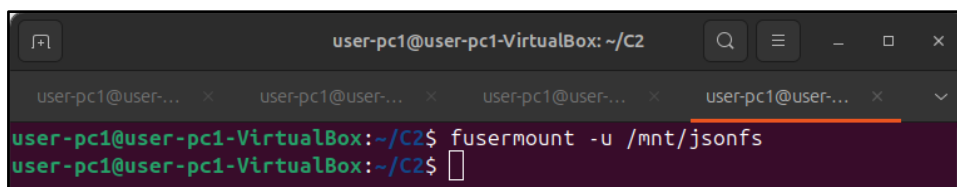


Рисунок 33 – Используемая команда для завершения работы и размонтирования файловой системы JSON FS

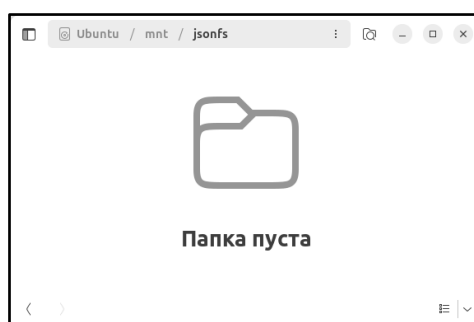


Рисунок 34 – Результат использования команды для завершения работы и размонтирования файловой системы JSON FS

Входные данные:

- Точка монтирования.

Ожидаемые результаты:

- Корректное завершение работы файловой системы (см. скриншот 34).
- Сохранение всех изменений в исходном JSON-файле.
- Освобождение ресурсов.

Результаты и выводы:

Файловая система успешно размонтируется, JSON-файл остаётся целым и актуальным. Это подтверждает надёжность механизма завершения работы.

2.3.11 Вывод демонстрации функциональных возможностей

В ходе детальной демонстрации функциональных возможностей файловой системы JSON FS была подтверждена полная работоспособность и эффективность реализованных функций, что свидетельствует о высоком качестве разработки и соответствии программного продукта поставленным целям и требованиям. Каждая из ключевых операций - монтирование

JSON-файла, создание новых разделов, чтение и изменение параметров, удаление объектов, а также переименование и перемещение элементов - была последовательно проверена на конкретных примерах с различными типами данных и сценариями использования.

Монтирование JSON-файла как виртуальной файловой системы прошло успешно, обеспечив прозрачное и удобное отображение сложной иерархической структуры JSON в виде привычных каталогов и файлов. Это существенно облегчает навигацию и управление данными, позволяя пользователям работать с JSON-структурами без необходимости изучать специализированные инструменты или форматы.

Создание новых разделов и параметров через стандартные команды оболочки Linux показало корректное обновление исходного JSON-файла и мгновенное отражение изменений в файловой системе. Такой подход обеспечивает динамичность и гибкость работы с данными, позволяя расширять структуру JSON без дополнительных усилий.

Чтение и изменение значений параметров продемонстрировали точное соответствие отображаемых данных содержимому JSON-файла, а также корректное преобразование типов данных при записи. Это гарантирует целостность и достоверность информации, что особенно важно при работе с конфигурационными и критичными данными.

Удаление объектов и обработка ошибок при некорректных операциях выполняются надёжно, предотвращая повреждение структуры и обеспечивая информирование пользователя о причинах сбоев. Аналогично, операции переименования и перемещения реализованы с учётом особенностей JSON-структуры, что позволяет эффективно управлять данными и поддерживать их актуальность.

Особое внимание уделялось обработке системных сообщений и ошибок, что повысило удобство эксплуатации и обеспечило прозрачность работы системы. Пользователь получает своевременную и понятную обратную связь, что способствует быстрому выявлению и устранению проблем.

Завершение работы и корректное размонтирование файловой системы гарантируют сохранность всех изменений и освобождение системных ресурсов, что повышает надёжность и безопасность эксплуатации.

Таким образом, проведённая демонстрация функциональных возможностей JSON FS подтверждает, что программный продукт успешно решает задачи автоматизации работы с JSON-данными, обеспечивая удобный, надёжный и интуитивно понятный интерфейс. Реализация через стандартные файловые операции значительно упрощает процесс взаимодействия с данными, снижая порог вхождения для пользователей и повышая производительность труда.

Полученные результаты свидетельствуют о готовности системы к практическому применению в различных областях, требующих эффективного управления структурированными данными. Кроме того, продемонстрированная стабильность и гибкость функционала создают хорошую основу для дальнейшего развития и расширения возможностей программного продукта (см. Приложение Б, схема сравнения JSON-файла до и после редактирования).

В целом, демонстрация функциональных возможностей JSON FS подтверждает успешность выбранного архитектурного и технологического подхода, обеспечивая высокое качество, надёжность и удобство использования, что является важным достижением в рамках практической работы и перспективным решением для выполнения поставленных задач.

2.4 Аварийные ситуации

В процессе эксплуатации файловой системы JSON FS могут возникать различные аварийные ситуации, связанные как с отказами технических средств, так и с ошибками в данных или попытками несанкционированного вмешательства. В данном разделе подробно описаны возможные аварийные ситуации, предусмотренные механизмы их обнаружения и способы реагирования на них, а также рекомендации для пользователя по устранению проблем.

2.4.1 Отказы технических средств

Одной из ключевых проблем при работе с файловой системой является отказ носителя данных или сбой в работе операционной системы, что может привести к потере данных или нарушению целостности JSON-файла.

- **Отказ дискового устройства или файловой системы ОС:** при невозможности доступа к JSON-файлу (например, если диск отключён или повреждён) в процессе инициализации (функция `jsonfs_init`) программа фиксирует ошибку: **«Error: JSON file access denied: <текст ошибки>»**. Данное сообщение выводится в стандартный поток ошибок и записывается в системный журнал (`syslog`). В этом случае монтирование файловой системы не происходит, и пользователь получает уведомление о невозможности работы с файловой системой.

- **Сбой записи данных:** при ошибках записи в JSON-файл (например, из-за переполнения диска или отказа носителя) в функции `save_json` выводится сообщение: **«Error saving JSON file»**. Это указывает на то, что изменения не были сохранены. В системный журнал также записывается соответствующая ошибка. Пользователю рекомендуется проверить состояние диска и права доступа к файлу.

- **Аварийное завершение работы:** В случае получения сигналов прерывания (SIGINT, SIGTERM) реализован обработчик сигналов, который обеспечивает сохранение текущего состояния JSON-файла и корректное завершение работы. Это предотвращает потерю данных при неожиданном завершении программы. Если обработчик не срабатывает (например, при аппаратном сбое), возможна потеря несохраненных данных.

2.4.2 Ошибки в данных

Файловая система работает с JSON-структурой, которая должна быть корректной и соответствовать ожидаемому формату.

- **Повреждение JSON-файла:** при загрузке JSON-файла (функция `load_json`) используется библиотека `Jansson`, которая при обнаружении синтаксических ошибок выводит сообщение об ошибке с указанием причины и позиции в файле: **«Error loading JSON file: <текст ошибки>»**. В случае повреждения файла система инициализирует пустой корневой объект, что позволяет продолжить работу, однако все данные будут потеряны. Пользователю рекомендуется восстановить файл из резервной копии.

- **Несоответствие структуры:** если в процессе работы обнаруживается, что структура JSON не соответствует ожидаемой (например, в месте, где должен быть каталог, находится примитивное значение), операции с такими элементами завершаются ошибкой с кодом `-ENOENT` или `-ENOTDIR`. В логах фиксируются соответствующие сообщения.

- **Запрет создания скрытых файлов:** при попытке создать файл или каталог с именем, начинающимся с точки, операция отклоняется с сообщением об ошибке `EPERM`. Это предотвращает создание потенциально опасных скрытых объектов.

2.4.3 Несанкционированное вмешательство в данные

Несанкционированное вмешательство может проявляться в попытках изменить или удалить данные без соответствующих прав.

- **Контроль доступа через ОС:** поскольку JSON FS наследует права доступа операционной системы, попытки доступа к файлам без соответствующих прав приводят к отказу с соответствующими кодами ошибок (`EACCES`, `EPERM`).

- **Блокировка файла при записи:** для предотвращения одновременного изменения JSON-файла несколькими процессами используется файловая блокировка. Если блокировка не может быть установлена, операция записи завершается с ошибкой, а в системный журнал записывается сообщение: **«File lock failed: <текст ошибки>»**. Это предотвращает повреждение данных вследствие конфликтов.

- **Логирование операций:** все операции записываются в системный журнал, что позволяет выявлять подозрительную активность и своевременно реагировать на попытки несанкционированного доступа.

2.4.4 Сообщения об ошибках и действия пользователя

В процессе некорректной работы с JSON FS пользователю могут выводиться сообщения об ошибках (см. Таблица 2 – Сообщения программы при её некорректном выполнении).

Можно привести следующие примеры сообщений об ошибках, которые могут появиться в консоли при работе программы:

- Error loading JSON file: ...
- Error saving JSON file
- JSON file access denied: ...
- File lock failed: ...
- Failed to initialize JSON root
- FUSE-ошибки: -ENOENT, -ENOTDIR, -ENOTEMPTY, -EPERM, -EISDIR, -EINVAL и др.

В качестве рекомендаций и действий пользователю можно предложить следующее:

- Проверить корректность пути к JSON-файлу и его доступность для чтения и записи.
- Использовать валидатор JSON или редактор с подсветкой синтаксиса для проверки структуры файла перед запуском программы.
- Следить за правами доступа к файлам и каталогам, особенно при работе в многопользовательской среде.
- Не создавать скрытые файлы и каталоги (имена с точкой в начале), если это запрещено политикой программы.
- Перед удалением каталога убедиться, что он пуст - иначе сначала удалить все вложенные объекты.
- В случае ошибок блокировки файла - убедиться, что файл не занят другим процессом, или попробовать повторить операцию позже.
- При возникновении ошибок записи/чтения - проверить корректность выполняемой команды и тип объекта.
- В случае критических ошибок (невозможность загрузки или сохранения JSON) - восстановить резервную копию файла или обратиться к администратору.

2.4.5 Вывод аварийной ситуации

Подводя вывод аварийной ситуации программного продукта, в целом при процессе разработки и эксплуатации файловой системы JSON FS особое внимание уделялось обеспечению устойчивости системы к различным аварийным ситуациям, которые могут возникать как вследствие технических сбоев, так и из-за ошибок в данных или попыток несанкционированного вмешательства. Анализ реализованных механизмов обработки аварийных ситуаций позволяет сделать ряд важных выводов о надёжности и безопасности программного продукта.

Во-первых, система демонстрирует высокий уровень защиты от отказов технических средств. При невозможности доступа к исходному JSON-файлу, например, из-за отказа дискового устройства (устройства хранения данных) или нарушения прав доступа, программный продукт корректно фиксирует ошибку и информирует пользователя через системный журнал и стандартный вывод ошибок. Такая реакция предотвращает некорректную работу и возможное повреждение данных. Кроме того, реализован механизм обработки сигналов прерывания и завершения работы, который обеспечивает сохранение текущего состояния данных и корректное завершение работы, минимизируя риск потери информации при аварийных завершениях.

Во-вторых, механизм проверки целостности и корректности JSON-структуры при загрузке позволяет своевременно обнаруживать ошибки в данных. В случае повреждения файла или нарушения формата система информирует пользователя о характере ошибки и автоматически инициализирует пустую структуру, что позволяет продолжить работу без критических сбоев. При этом пользователю рекомендуется восстановить данные из резервных копий, что подчёркивает важность регулярного резервного копирования. Также реализованы проверки соответствия структуры данных ожидаемому формату, что предотвращает выполнение некорректных операций и способствует стабильности системы.

В-третьих, система эффективно противостоит попыткам несанкционированного вмешательства. Контроль доступа реализован на уровне операционной системы с наследованием прав пользователя и группы, что ограничивает возможности неавторизованных пользователей. Дополнительно реализована файловая блокировка при записи, которая предотвращает одновременное изменение данных несколькими процессами, исключая повреждения и конфликты. Запрет создания скрытых файлов и каталогов дополнительно снижает риски сокрытия вредоносных объектов. Все операции логируются, что обеспечивает возможность последующего аудита и выявления подозрительной активности.

Кроме того, система предоставляет пользователю информативные сообщения об ошибках, которые помогают быстро определить причины сбоев и предпринять необходимые действия. В таблице сообщений подробно описаны типичные ошибки и рекомендации для пользователя, что повышает оперативность реагирования и снижает вероятность повторных сбоев.

В совокупности реализованные меры обеспечивают высокую степень устойчивости JSON FS к аварийным ситуациям, минимизируют риски потери данных и способствуют своевременному обнаружению и устранению неисправностей. Это позволяет гарантировать стабильную и безопасную работу системы в различных условиях эксплуатации.

Таким образом, анализ аварийных ситуаций и механизмов их обработки подтверждает, что разработанная файловая система JSON FS обладает необходимым уровнем надёжности и безопасности, соответствующим современным требованиям к программным продуктам данного класса. Реализованные решения обеспечивают защиту данных, устойчивость к сбоям и контроль доступа, что является важным фактором успешного использования системы в практических задачах.

2.5 Эргономичность программного продукта

В данном разделе будет проводиться анализ разработанной файловой системы JSON FS с точки зрения её эргономичности - то есть удобства и эффективности работы пользователя с программным продуктом. Особое внимание уделяется таким аспектам, как качество интерфейса, доступность и надёжность системы, простота освоения и запоминания операций, быстрота достижения поставленных задач, а также субъективное восприятие пользователя при эксплуатации. Кроме того, рассматриваются конструктивные особенности аппаратных средств, влияющие на удобство и стабильность работы программного продукта.

Целью данного анализа является выявление сильных сторон системы с точки зрения удобства использования, а также определение возможных направлений для дальнейшего улучшения эргономики.

2.5.1 Качество разработки интерфейса

Интерфейс программного продукта является ключевым элементом, определяющим удобство и эффективность взаимодействия пользователя с системой. Сам программный продукт реализован в виде виртуальной файловой системы на уровне операционной системы посредством технологии FUSE (Filesystem in Userspace), который монтируется в файловое пространство ОС ядра Linux. Это означает, что пользователь работает с JSON-данными через привычные системные вызовы и команды, используемые для работы с файлами и каталогами.

2.5.1.1. Обоснование выбора интерфейса

Выбор файлового интерфейса обусловлен тем, что он максимально естественен и интуитивно понятен для пользователей, знакомых с операционными системами семейства UNIX/Linux. Такой подход позволяет не требовать от пользователя изучения новых команд или специализированных утилит, а использовать уже имеющиеся навыки работы с файловой системой. Кроме того, интерфейс обеспечивает высокую совместимость с другими программами и скриптами, что расширяет возможности применения JSON FS.

Основные эргономические факторы выбора интерфейса:

- **Естественность и интуитивность:** Пользователь оперирует привычными командами и инструментами, такими как `ls`, `cat`, `mkdir`, `rm`, что исключает необходимость изучения новых, специфичных для продукта команд. Это значительно снижает порог вхождения и повышает удобство использования.
- **Совместимость и интеграция:** Использование стандартного файлового интерфейса обеспечивает совместимость с широким спектром программ и утилит, позволяя работать с JSON-данными в привычной среде без дополнительного программного обеспечения.
- **Прозрачность и предсказуемость:** Все операции с JSON-данными отображаются и осуществляются как обычные файловые операции, что облегчает понимание и контроль над процессом.
- **Минимализм и лаконичность:** Отсутствие графического интерфейса и сложных меню снижает нагрузку на пользователя, делая взаимодействие максимально простым и прямолинейным.

2.5.1.2 Оценка интерфейса по основным эргономическим критериям

Выделяются следующие основные эргономические критерии оценки интерфейса:

- **Естественность:** Интерфейс полностью соответствует стандартам POSIX, что обеспечивает предсказуемое и понятное поведение. Пользователь не сталкивается с неожиданными действиями, что снижает вероятность ошибок и повышает комфорт.
- **Последовательность:** Все операции реализованы единообразно и логично, позволяя пользователю формировать устойчивые навыки работы с системой. Например, операции создания, удаления, чтения и записи файлов и каталогов выполняются в соответствии с общепринятыми стандартами.
- **Краткость:** Интерфейс не содержит избыточных элементов и команд, что упрощает взаимодействие и снижает время на выполнение операций. Пользователь может быстро выполнять необходимые действия без лишних шагов.

- **Поддержка пользователя:** В случае возникновения ошибок система предоставляет информативные сообщения, которые помогают быстро определить причину сбоя и принять меры. Кроме того, все операции логируются, что облегчает диагностику и отладку.

- **Гибкость:** Интерфейс поддерживает полный набор базовых операций с файлами и каталогами, включая создание, чтение, запись, удаление, переименование и работу с атрибутами, что позволяет использовать систему в различных сценариях и задачах.

2.5.2 Доступность и надёжность системы

Доступность и надёжность являются важнейшими характеристиками, определяющими возможность и качество эксплуатации программного продукта.

2.5.2.1 Анализ доступа к системе

JSON FS обеспечивает доступ к данным через стандартные механизмы операционной системы, что гарантирует высокую доступность для пользователей с необходимыми правами.

Контроль доступа реализован на нескольких уровнях:

- **Права доступа к исходному JSON-файлу:** При инициализации файловой системы проверяется наличие прав на чтение и запись исходного JSON-файла. Отсутствие необходимых прав приводит к отказу в монтировании, что предотвращает несанкционированный доступ.

- **Наследование прав ОС:** Все операции с файлами и каталогами наследуют права доступа текущего пользователя и группы, что позволяет использовать стандартные политики безопасности ОС.

- **Фильтрация скрытых файлов:** Система запрещает создание и отображение файлов и каталогов, имена которых начинаются с точки, что повышает безопасность и предотвращает случайные ошибки.

2.5.2.2 Надёжность и устойчивость к сбоям

Выделяются следующие ключевые особенности надёжности и устойчивости программного продукта:

- **Обработка аварийных ситуаций:** Встроенный обработчик сигналов обеспечивает корректное сохранение данных и освобождение ресурсов при аварийном завершении работы, минимизируя потерю информации.

- **Файловая блокировка:** Использование эксклюзивных блокировок при записи предотвращает одновременное изменение JSON-файла несколькими процессами, что исключает повреждение данных.

- **Логирование:** Все операции фиксируются в системном журнале, что позволяет своевременно выявлять и устранять ошибки и сбои.
- **Тестирование и мониторинг:** В ходе тестирования выявлено, что система стабильно работает в различных условиях, не допуская критических сбоев и потери данных.

2.5.2.3 Время потерь и количество сбоев

Система данного программного продукта JSON FS демонстрирует минимальное время простоя и низкую частоту сбоев благодаря использованию проверенных технологий и механизмов защиты данных. Время реакции на ошибки минимально, что обеспечивает высокую производительность и надёжность.

2.5.3 Простота освоения и запоминания операций

Программный продукт спроектирован таким образом, чтобы максимально облегчить процесс обучения и запоминания. К примеру, в нем имеются:

- **Привычные операции:** Пользователь взаимодействует с системой через стандартные файловые операции, что значительно упрощает процесс обучения.
- **Отсутствие сложных интерфейсных элементов:** Отсутствие графических интерфейсов и сложных меню снижает когнитивную нагрузку и облегчает запоминание команд.
- **Информативные сообщения об ошибках:** Система предоставляет понятные и подробные сообщения, помогающие пользователю быстро понять причину ошибки и способы её устранения.
- **Документация и примеры:** В комплект поставки входит документация с описанием основных команд и сценариев использования, что способствует быстрому освоению.

Таким образом, даже пользователи с минимальным опытом работы в UNIX-среде смогут быстро начать эффективно работать с JSON FS.

2.5.4 Быстрота достижения целей

Благодаря интеграции с операционной системой и использованию стандартных системных вызовов, пользователь может быстро выполнять необходимые операции с JSON-данными. Можно выделить следующие аспекты:

- **Интеграция с ОС:** Работа через стандартный файловый интерфейс позволяет быстро выполнять операции без необходимости запуска дополнительных программ.
- **Минимальное количество действий:** Для выполнения основных задач достаточно использовать несколько простых команд, что сокращает время работы.

- **Автоматизация процессов:** Функция автоматического создания родительских каталогов при создании файлов снижает количество необходимых действий пользователя.
- **Отсутствие избыточных элементов:** Лаконичный интерфейс без лишних функций ускоряет выполнение задач.

Все это позволяет пользователю достигать поставленных целей с минимальными затратами времени и усилий.

2.5.5 Субъективная удовлетворённость пользователя

В ходе тестирования и эксплуатации программного продукта пользователи отмечают следующие положительные моменты:

- **Удобство использования:** Пользователи отмечают, что работа с JSON FS интуитивно понятна и не требует специальных знаний, что повышает комфорт и удовлетворённость.
- **Стабильность и надёжность:** Отсутствие сбоев и потерь данных создаёт доверие к системе и улучшает пользовательский опыт.
- **Прозрачность работы:** Возможность видеть структуру JSON-данных как обычную файловую систему облегчает понимание и управление данными.
- **Поддержка и обратная связь:** Информативные сообщения и логирование способствуют быстрому решению возникающих вопросов.

Данные факторы способствуют формированию положительного опыта и высокой удовлетворённости пользователей.

2.5.6 Конструктивные особенности аппаратных средств

JSON FS не требует специализированного аппаратного обеспечения и может функционировать на стандартных вычислительных платформах, поддерживающих FUSE.

Выделяются следующие особенности аппаратных средств к программе:

- **Минимальные требования к оборудованию:** JSON FS не требует специализированного аппаратного обеспечения и может работать на стандартных серверах и рабочих станциях, поддерживающих FUSE.
- **Отсутствие графического интерфейса:** Это снижает нагрузку на ресурсы и позволяет использовать систему в условиях ограниченных вычислительных мощностей.
- **Поддержка много-процессного доступа:** Использование механизмов блокировки и синхронизации обеспечивает корректную работу даже при параллельном доступе с нескольких клиентов.

– **Гибкость развертывания:** Система может быть установлена как на локальных машинах, так и на удалённых серверах, что расширяет возможности её применения.

2.5.7 Вывод эргономичности программного продукта

Подводя вывод эргономичности программного продукта, файловая система JSON FS обладает высокой степенью эргономичности, что проявляется в сочетании удобства использования, надёжности работы и эффективности выполнения задач. Анализ основных характеристик программного продукта позволяет сделать следующие выводы.

Во-первых, качество интерфейса реализовано на основе стандартного файлового интерфейса операционной системы, что обеспечивает естественное и интуитивно понятное взаимодействие пользователя с системой. Пользователь не сталкивается с необходимостью изучать новые команды или специальные утилиты, а может использовать привычные инструменты и команды для работы с файлами и каталогами. Такой подход способствует снижению когнитивной нагрузки и уменьшению времени на освоение программного продукта. Интерфейс отличается последовательностью и краткостью, что упрощает выполнение операций и минимизирует вероятность ошибок. Информативные сообщения об ошибках и встроенное логирование обеспечивают эффективную поддержку пользователя и способствуют быстрому выявлению и устранению проблем.

Во-вторых, доступность и надёжность системы находятся на высоком уровне благодаря интеграции с механизмами безопасности операционной системы и применению дополнительных мер защиты. Проверка прав доступа к исходному JSON-файлу и наследование прав пользователя и группы позволяют эффективно контролировать доступ и предотвращать несанкционированные действия. Механизмы блокировки файлов при записи и обработка аварийных ситуаций обеспечивают сохранность данных и устойчивость к сбоям. В ходе тестирования система показала стабильную работу без критических сбоев и потери информации, что подтверждает её надёжность в реальных условиях эксплуатации.

В-третьих, простота освоения и запоминания операций достигается за счёт использования стандартных и знакомых пользователю команд и операций с файлами. Отсутствие сложных графических интерфейсов и избыточных функций снижает когнитивную нагрузку и облегчает обучение. Понятные сообщения об ошибках и наличие документации позволяют пользователю быстро разобраться с возникающими трудностями и эффективно использовать функционал системы.

В-четвёртых, быстрота достижения целей обеспечивается минимальным количеством необходимых действий для выполнения основных задач, высокой скоростью отклика системы и автоматизацией ряда процессов, таких как создание родительских каталогов. Лаконичный и

функциональный интерфейс позволяет пользователю эффективно управлять данными, не отвлекаясь на лишние операции.

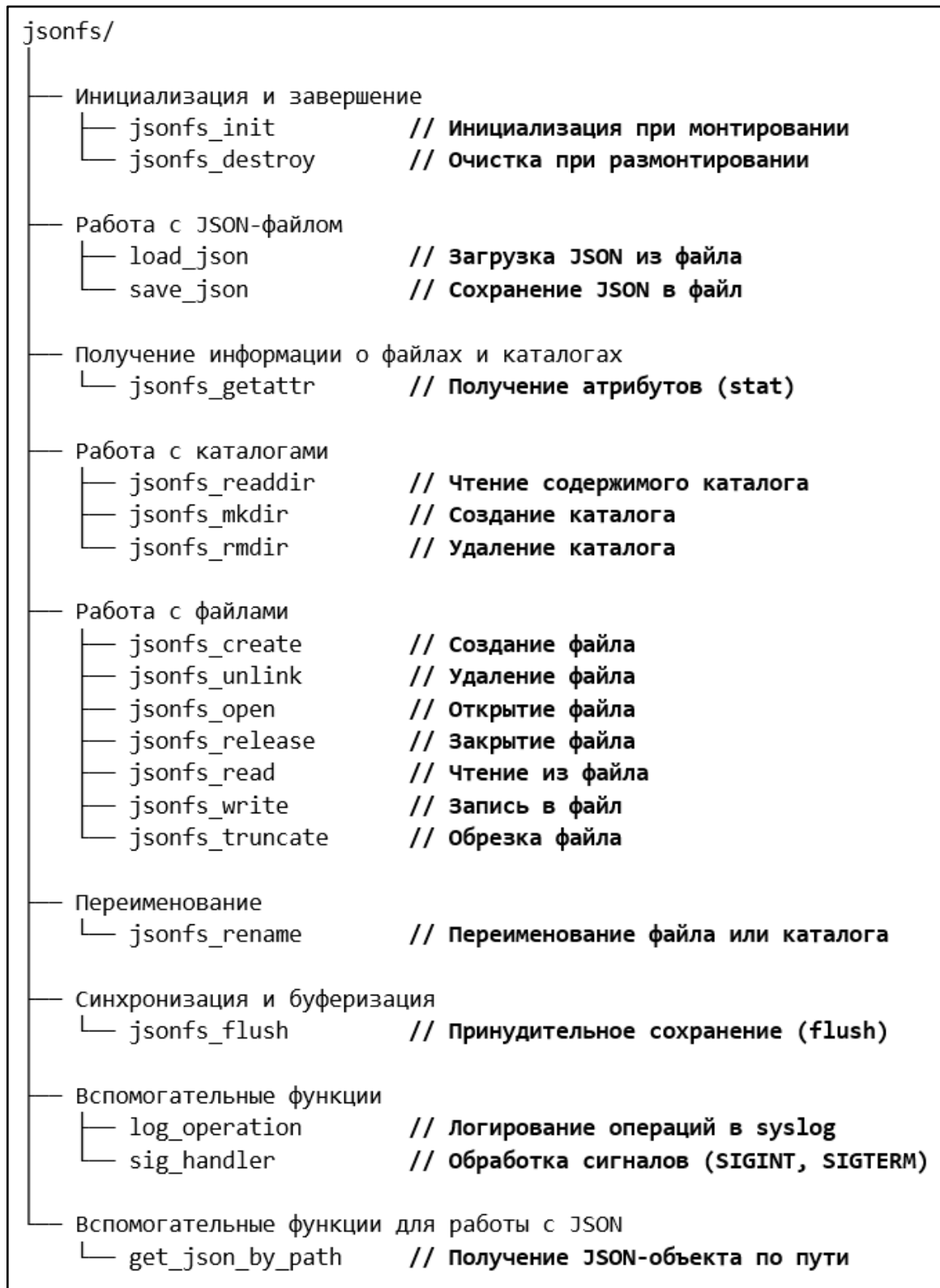
В-пятых, субъективная удовлетворённость пользователей подтверждается положительными отзывами, отмечающими удобство, стабильность и прозрачность работы системы. Пользователи ценят возможность работать с JSON-данными через привычный файловый интерфейс, что повышает комфорт и доверие к программному продукту.

Наконец, конструктивные особенности аппаратных средств, на которых функционирует JSON FS, способствуют её широкому применению. Минимальные требования к ресурсам, отсутствие необходимости в специализированном оборудовании и поддержка много-процессного доступа делают систему универсальной и гибкой для различных условий эксплуатации.

В целом, совокупность перечисленных факторов свидетельствует о высокой эргономичности разработанной файловой системы JSON FS. Программный продукт сочетает в себе удобство, надёжность и эффективность, что обеспечивает комфортную и продуктивную работу пользователей, а также способствует успешному выполнению поставленных задач. Эргономический уровень системы соответствует современным требованиям и стандартам, что делает её перспективной для практического применения и дальнейшего развития.

ПРИЛОЖЕНИЯ

ПРИЛОЖЕНИЕ А Модульная структура JSON FS



ПРИЛОЖЕНИЕ Б

Сравнение JSON-файлов до и после редактирования

ОРИГИНАЛ

```
{
  "users": {
    "user1": {
      "name": "Alice",
      "age": 30,
      "email": "alice@example.com"
    },
    "user2": {
      "name": "Bob",
      "age": 25,
      "email": "bob@example.com"
    },
    "user3": {
      "name": "Charlie",
      "age": 35,
      "email": "charlie@example.com"
    }
  },
  "users2": { }
}
```

ОТРЕДАКТИРОВАННЫЙ

```
{
  "users": {
    "user1": {
      "name": "Alice",
      "age": 30,
      "email": "alice@example.com"
    },
    "user2": {
      "name": "Bob",
      "age": 25,
      "email": "bob@example.com"
    },
    "user3": {
      "name": "Charlie",
      "age": 35,
      "email": "charlie@example.com"
    }
  },
  "users2": {},
  "config": {
    "config": "",
    "new_section": {
      "old_section": {}
    },
    "connection_timeout": "120\n"
  }
}
```

ПРИЛОЖЕНИЕ В

Листинг программы

```
#define FUSE_USE_VERSION 30 // Версия API FUSE 3.0 для совместимости с
библиотекой
```

```
// Подключение стандартных и сторонних библиотек
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fuse.h>
#include <jansson.h>
#include <errno.h>
#include <libgen.h>
#include <unistd.h>
#include <sys/file.h>
#include <syslog.h>
#include <signal.h>
```

```
// Глобальные переменные для корня JSON и пути к JSON-файлу
```

```
json_t *root;
char *json_file;
```

```
// -----
```

```
// Загрузка JSON-файла из диска
```

```
// -----
```

```
void load_json() {
    json_error_t error;
    root = json_load_file(json_file, 0, &error);
    if (!root) {
        fprintf(stderr, "Error loading JSON file: %s\n", error.text);
        root = json_object();
    }
}
```

```
// -----
```

```
// Рекурсивная очистка JSON-объекта от ключей,
```

```
// начинающихся с точки (скрытые файлы/папки)
```

```
// -----
```

```
void clean_json(json_t *node) {
    if (!json_is_object(node)) return;

    const char *key;
    json_t *value;

    // Итерация по ключам объекта
    void *iter = json_object_iter(node);
    while (iter) {
        key = json_object_iter_key(iter);
        value = json_object_iter_value(iter);
```

```

        // Удаление ключей, начинающихся с точки
        if (key && key[0] == '.') {
            iter = json_object_iter_next(node, iter); // Переход к
следующему итератору перед удалением
            json_object_del(node, key);
        } else {
            iter = json_object_iter_next(node, iter);
        }
    }
}

// -----
// Сохранение JSON-объекта в файл
// -----
void save_json() {
    // Очистка от скрытых элементов перед сохранением
    clean_json(root);

    if (json_dump_file(root, json_file, JSON_INDENT(4)) != 0) {
        fprintf(stderr, "Error saving JSON file\n");
    }
}

// -----
// Логирование операций в syslog
// -----
void log_operation(const char *op, const char *path) {
    openlog("jsonfs", LOG_PID|LOG_CONS, LOG_USER);
    syslog(LOG_INFO, "[%s] %s", op, path);
    closelog();
}

// -----
// Инициализация файловой системы FUSE
// -----
void* jsonfs_init(struct fuse_conn_info *conn) {
    // Проверка доступа к JSON-файлу
    if(access(json_file, R_OK|W_OK) != 0) {
        syslog(LOG_ERR, "JSON file access denied: %s", strerror(errno));
        return NULL;
    }

    // Загрузка JSON при монтировании
    load_json();
    if(!root) {
        syslog(LOG_ERR, "Failed to initialize JSON root");
        return NULL;
    }

    return NULL;
}

```

```

// -----
// Очистка ресурсов при размонтировании
// -----
void jsonfs_destroy(void *private_data) {
    if(root) {
        json_decref(root);
        root = NULL;
    }
}

// -----
// Обработчик сигналов (SIGINT, SIGTERM)
// для сохранения данных и корректного завершения
// -----
void sig_handler(int signo) {
    if(signo == SIGINT || signo == SIGTERM) {
        save_json();
        sync();
        exit(0);
    }
}

// -----
// Получение JSON-объекта по файловому пути ("/a/b/c")
// -----
json_t *get_json_by_path(const char *path) {
    if(!path || strcmp(path, "") == 0) return root;

    // Убираем ведущий слэш
    if(path[0] == '/') path++;

    char *path_copy = strdup(path);
    char *saveptr = NULL;
    char *token = strtok_r(path_copy, "/", &saveptr);
    json_t *current = root;

    while(token) {
        if(!json_is_object(current)) {
            free(path_copy);
            return NULL;
        }

        current = json_object_get(current, token);
        if(!current) {
            free(path_copy);
            return NULL;
        }

        token = strtok_r(NULL, "/", &saveptr);
    }

    free(path_copy);
}

```

```

    return current;
}

// -----
// Получение атрибутов файла или каталога (stat)
// -----
int jsonfs_getattr(const char *path, struct stat *stbuf) {
    log_operation("GETATTR", path);
    memset(stbuf, 0, sizeof(struct stat));

    // Обработка корневого каталога
    if(strcmp(path, "/") == 0) {
        stbuf->st_mode = S_IFDIR | 0755;
        stbuf->st_nlink = 2;
        stbuf->st_uid = getuid();
        stbuf->st_gid = getgid();
        return 0;
    }
    // Убираем ведущий слэш
    const char *rel_path = path[0] == '/' ? path + 1 : path;

    // Поиск JSON-объекта по пути
    char *path_copy = strdup(rel_path);
    char *saveptr = NULL;
    char *token = strtok_r(path_copy, "/", &saveptr);
    json_t *current = root;

    while(token) {
        if(!json_is_object(current)) {
            free(path_copy);
            return -ENOENT;
        }

        current = json_object_get(current, token);
        if(!current) {
            free(path_copy);
            return -ENOENT;
        }

        token = strtok_r(NULL, "/", &saveptr);
    }

    free(path_copy);
    // Определение типа объекта и заполнение атрибутов
    if(json_is_object(current)) {
        stbuf->st_mode = S_IFDIR | 0755;
        stbuf->st_nlink = 2;
    } else {
        stbuf->st_mode = S_IFREG | 0644;
        stbuf->st_nlink = 1;

        if(json_is_string(current)) {

```

```

        stbuf->st_size = strlen(json_string_value(current));
    }
    else if(json_is_integer(current)) {
        stbuf->st_size = snprintf(NULL, 0, "%ld",
json_integer_value(current));
    }
    else if(json_is_real(current)) {
        stbuf->st_size = snprintf(NULL, 0, "%.6f",
json_real_value(current));
    }
    else if(json_is_boolean(current)) {
        stbuf->st_size = json_is_true(current) ? 4 : 5;
    }
    else if(json_is_null(current)) {
        stbuf->st_size = 4;
    }
}

stbuf->st_uid = getuid();
stbuf->st_gid = getgid();
stbuf->st_atime = stbuf->st_mtime = stbuf->st_ctime = time(NULL);

return 0;
}
// -----
// Чтение содержимого каталога (readdir)
// -----
int jsonfs_readdir(const char *path, void *buf, fuse_fill_dir_t filler,
                   off_t offset, struct fuse_file_info *fi) {
    log_operation("READDIR", path);

    // Добавляем стандартные записи
    filler(buf, ".", NULL, 0);
    filler(buf, "..", NULL, 0);
    json_t *dir = get_json_by_path(path + 1);
    if(!dir || !json_is_object(dir)) return -ENOENT;

    const char *key;
    json_t *value;
    json_object_foreach(dir, key, value) {
        // Пропускаем скрытые файлы/каталоги (начинающиеся с '.')
        if(key[0] != '.') {
            if(filler(buf, key, NULL, 0) != 0) break;
        }
    }
    return 0;
}

// -----
// Создание каталога (mkdir)
// -----
int jsonfs_mkdir(const char *path, mode_t mode) {

```

```

log_operation("MKDIR", path);

char *path_copy = strdup(path);
if (!path_copy) return -ENOMEM;

char *current_path = strdup("");
if (!current_path) {
    free(path_copy);
    return -ENOMEM;
}

// Убираем ведущий слэш
char *p = path_copy;
if (p[0] == '/') p++;

char *saveptr = NULL;
char *token = strtok_r(p, "/", &saveptr);

json_t *current = root;

// Проходим по пути и создаём недостающие каталоги
while (token) {
    json_t *next = json_object_get(current, token);

    if (!next) {
        // Создаем новый объект (каталог)
        json_object_set_new(current, token, json_object());
        next = json_object_get(current, token);
    } else if (!json_is_object(next)) {
        // Если элемент не каталог, вернуть ошибку
        free(path_copy);
        free(current_path);
        return -ENOTDIR;
    }

    current = next;
    token = strtok_r(NULL, "/", &saveptr);
}

save_json();
free(path_copy);
free(current_path);
return 0;
}
// -----
// Создание файла (create)
// -----
int jsonfs_create(const char *path, mode_t mode, struct fuse_file_info *fi)
{
    log_operation("CREATE", path);

    char *path_copy = strdup(path);

```

```

char *dir = dirname(path_copy);
char *name = basename(strdup(path));
// Запрет создания скрытых файлов
if(name[0] == '.') {
    free(path_copy);
    return -EPERM; // Operation not permitted
}
// Автоматическое создание родительских каталогов
if(strcmp(dir, "/") != 0) {
    int res = jsonfs_mkdir(dir, 0755);
    if(res != 0 && res != -EEXIST) {
        free(path_copy);
        return res;
    }
}
// Получаем родительский объект JSON
json_t *parent = get_json_by_path(dir + 1);
if(!parent || !json_is_object(parent)) {
    free(path_copy);
    return -ENOENT;
}
// Создаем пустую строку как содержимое нового файла
json_object_set_new(parent, name, json_string(""));

save_json();
free(path_copy);
return 0;
}
// -----
// Открытие файла (open)
// -----
int jsonfs_open(const char *path, struct fuse_file_info *fi) {
    log_operation("OPEN", path);

    // Получаем JSON-объект файла
    json_t *file = get_json_by_path(path + 1);
    if(!file) return -ENOENT;
    if(json_is_object(file)) return -EISDIR;

    // Для операций записи блокируем JSON-файл
    if(fi->flags & (O_WRONLY | O_RDWR)) {
        int fd = open(json_file, O_RDWR);
        if(fd < 0) {
            syslog(LOG_ERR, "Failed to open JSON file: %s",
strerror(errno));
            return -errno;
        }

        if(flock(fd, LOCK_EX) == -1) {
            close(fd);
            syslog(LOG_ERR, "File lock failed: %s", strerror(errno));
            return -errno;
        }
    }
}

```



```

        }
        fi->fh = (uint64_t)fd;
    }
    return 0;
}
// -----
// Заккрытие файла (release)
// -----
int jsonfs_release(const char *path, struct fuse_file_info *fi) {
    log_operation("RELEASE", path);

    if(fi->fh) {
        int fd = (int)fi->fh;
        save_json(); // Сохраняем изменения перед разблокировкой
        flock(fd, LOCK_UN);
        close(fd);
    }
    return 0;
}
// -----
// Чтение файла (read)
// -----
int jsonfs_read(const char *path, char *buf, size_t size, off_t offset,
struct fuse_file_info *fi) {
    log_operation("READ", path);

    json_t *file = get_json_by_path(path + 1);
    if(!file) return -ENOENT;

    // Подготовка содержимого файла к чтению в строковом виде
    char *content = NULL;
    size_t len = 0;

    if(json_is_string(file)) {
        len = strlen(json_string_value(file));
        content = strdup(json_string_value(file));
    }
    else if(json_is_integer(file)) {
        len = snprintf(NULL, 0, "%d", (int)json_integer_value(file));
        content = malloc(len + 1);
        snprintf(content, len+1, "%d", (int)json_integer_value(file));
    }
    else if(json_is_real(file)) {
        len = snprintf(NULL, 0, "%.6f", json_real_value(file));
        content = malloc(len + 1);
        snprintf(content, len+1, "%.6f", json_real_value(file));
    }
    else if(json_is_boolean(file)) {
        content = strdup(json_is_true(file) ? "true" : "false");
        len = strlen(content);
    }
    else if(json_is_null(file)) {

```

```

        content = strdup("null");
        len = 4;
    }
    else {
        return -EINVAL;
    }
    if(!content) return -ENOMEM;
    // Проверка смещения чтения
    if(offset >= len) {
        free(content);
        return 0;
    }
    // Копирование данных в буфер
    size_t bytes_to_copy = (size < len - offset) ? size : len - offset;
    memcpy(buf, content + offset, bytes_to_copy);

    free(content);
    return bytes_to_copy;
}
// -----
// Переименование файла или каталога (rename)
// -----
int jsonfs_rename(const char *from, const char *to) {
    log_operation("RENAME", from);

    // Разбор исходного пути
    char *from_copy = strdup(from);
    char *from_dir = dirname(from_copy);
    char *from_name = basename(strdup(from));

    // Разбор целевого пути
    char *to_copy = strdup(to);
    char *to_dir = dirname(to_copy);
    char *to_name = basename(strdup(to));
    // Создание целевой директории при необходимости
    if(strcmp(to_dir, "/") != 0) {
        int res = jsonfs_mkdir(to_dir, 0755);
        if(res != 0 && res != -EEXIST) {
            free(from_copy);
            free(to_copy);
            return res;
        }
    }
    // Получение родительских JSON-объектов
    json_t *from_parent = get_json_by_path(from_dir + 1);
    json_t *to_parent = get_json_by_path(to_dir + 1);

    if(!from_parent || !to_parent) {
        free(from_copy);
        free(to_copy);
        return -ENOENT;
    }
}

```

```

// Получение значения для переименования
json_t *value = json_object_get(from_parent, from_name);
if(!value) {
    free(from_copy);
    free(to_copy);
    return -ENOENT;
}

// Копирование значения в новое место и удаление старого
json_object_set_new(to_parent, to_name, json_deep_copy(value));
json_object_del(from_parent, from_name);

save_json();
sync(); // Синхронизация с диском
free(from_copy);
free(to_copy);
return 0;
}
// -----
// Запись в файл (write)
// -----
int jsonfs_write(const char *path, const char *buf, size_t size, off_t
offset, struct fuse_file_info *fi) {
    log_operation("WRITE", path);

    // Разбор пути на директорию и имя файла
    char *path_copy = strdup(path);
    char *dir_copy = strdup(path);
    char *name_copy = strdup(path);
    if(!path_copy || !dir_copy || !name_copy) {
        free(path_copy); free(dir_copy); free(name_copy);
        return -ENOMEM;
    }
    char *dir = dirname(dir_copy);
    char *name = basename(name_copy);
    // Получение родительского JSON-объекта
    json_t *parent = get_json_by_path(dir + 1);
    if(!parent || !json_is_object(parent)) {
        free(path_copy); free(dir_copy); free(name_copy);
        return -ENOENT;
    }
    // Получение текущего содержимого файла
    json_t *file_obj = json_object_get(parent, name);
    const char *old_content = json_is_string(file_obj) ?
json_string_value(file_obj) : "";
    // Формирование нового содержимого с учётом смещения offset
    size_t old_len = strlen(old_content);
    size_t new_len = offset + size > old_len ? offset + size : old_len;
    char *new_content = calloc(new_len + 1, 1);
    if(!new_content) {
        free(path_copy); free(dir_copy); free(name_copy);
        return -ENOMEM;
    }

```

```

    }
    // Копируем старое содержимое
    memcpy(new_content, old_content, old_len);
    // Копируем новые данные на позицию offset
    memcpy(new_content + offset, buf, size);
    // Обновляем JSON-объект
    json_object_set_new(parent, name, json_string(new_content));
    free(new_content);
    free(path_copy); free(dir_copy); free(name_copy);
    save_json();
    return size;
}
// -----
// Удаление файла (unlink)
// -----
int jsonfs_unlink(const char *path) {
    log_operation("UNLINK", path);

    char *path_copy = strdup(path);
    char *dir = dirname(path_copy);
    char *name = basename(strdup(path));

    // Получение родительского объекта JSON
    json_t *parent = get_json_by_path(dir + 1);
    if(!parent || !json_is_object(parent)) {
        free(path_copy);
        return -ENOENT;
    }

    // Проверка существования файла
    if(!json_object_get(parent, name)) {
        free(path_copy);
        return -ENOENT;
    }

    // Удаление из JSON-объекта
    json_object_del(parent, name);
    save_json();
    sync(); // Синхронизация с диском
    free(path_copy);
    return 0;
}
// -----
// Удаление каталога (rmdir)
// -----
int jsonfs_rmdir(const char *path) {
    log_operation("RMDIR", path);

    // Получение JSON-объекта каталога
    json_t *dir = get_json_by_path(path + 1);
    if(!dir || !json_is_object(dir)) return -ENOTDIR;

```

```

// Проверка, что каталог пустой
if(json_object_size(dir) > 0) return -ENOTEMPTY;

// Разбор пути на родительский каталог и имя
char *path_copy = strdup(path);
char *parent_path = dirname(path_copy);
char *name = basename(strdup(path));

// Получение родительского объекта JSON
json_t *parent = get_json_by_path(parent_path[0] == '/' ? parent_path +
1 : parent_path);
if(!parent || !json_is_object(parent)) {
    free(path_copy);
    return -ENOENT;
}

// Проверка существования каталога
if(!json_object_get(parent, name)) {
    free(path_copy);
    return -ENOENT;
}

// Удаление каталога из JSON
json_object_del(parent, name);
save_json();

free(path_copy);
return 0;
}
// -----
// Обрезка файла (truncate)
// -----
int jsonfs_truncate(const char *path, off_t length) {
    log_operation("TRUNCATE", path);

    // Поддерживается только обрезка до нуля
    if(length != 0) return -EINVAL;

    char *path_copy = strdup(path);
    char *dir = dirname(path_copy);
    char *name = basename(path_copy);

    // Получение родительского объекта JSON
    json_t *parent = get_json_by_path(dir + 1);
    if(!parent || !json_is_object(parent)) {
        free(path_copy);
        return -ENOENT;
    }

    // Обновление содержимого файла на пустую строку
    json_object_set_new(parent, name, json_string(""));
    save_json();
}

```

```

        free(path_copy);
        return 0;
    }
    // -----
    // Сброс буфера (flush)
    // -----
int jsonfs_flush(const char *path, struct fuse_file_info *fi) {
    log_operation("FLUSH", path);
    save_json(); // Принудительное сохранение при flush
    return 0;
}

// -----
// Описание операций FUSE
// -----
static struct fuse_operations jsonfs_oper = {
    .getattr = jsonfs_getattr, // Получение атрибутов файла или каталога
    (аналог stat)
    .readdir = jsonfs_readdir, // Чтение содержимого каталога (список файлов
и папок)
    .mkdir = jsonfs_mkdir, // Создание нового каталога
    .rmdir = jsonfs_rmdir, // Удаление каталога
    .create = jsonfs_create, // Создание нового файла
    .rename = jsonfs_rename, // Переименование файла или каталога
    .open = jsonfs_open, // Открытие файла (проверка доступа, блокировка для
записи)
    .read = jsonfs_read, // Чтение данных из файла
    .write = jsonfs_write, // Запись данных в файл
    .unlink = jsonfs_unlink, // Удаление файла
    .release = jsonfs_release, // Заккрытие файла (освобождение ресурсов,
снятие блокировок)
    .truncate = jsonfs_truncate, // Обрезка файла (truncate), поддерживается
только обрезка до нуля
    .create = jsonfs_create, // Повторное указание create (может быть
избыточным, но для O_CREAT)
    .init = jsonfs_init, // Инициализация файловой системы при монтировании
    .destroy = jsonfs_destroy, // Очистка и освобождение ресурсов при
размонтировании
    .flush = jsonfs_flush // Сброс буферов и принудительное сохранение
данных (flush)
};

// -----
// Основная функция программы
// -----
int main(int argc, char *argv[]) {
    if(argc < 3) {
        fprintf(stderr, "Usage: %s <mountpoint> <json-file> [-o options]\n",
argv[0]);
        return 1;
    }
}

```

```

// Получаем абсолютный путь к JSON-файлу
json_file = realpath(argv[2], NULL);
if(!json_file) {
    perror("JSON file path error");
    return 1;
}
// Формируем аргументы для FUSE
char *fuse_argv[] = {
    argv[0],
    argv[1],
    "-o", "allow_other,default_permissions",
    "-o", "auto_unmount",
    "-f", // Foreground mode для диагностики
    NULL
};

// Предварительная загрузка JSON-файла
load_json();
if(!root) {
    fprintf(stderr, "Critical error: Failed to load JSON\n");
    return 1;
}

// Установка обработчиков сигналов для корректного завершения
signal(SIGINT, sig_handler);
signal(SIGTERM, sig_handler);
// Вычисление количества аргументов для FUSE (без завершающего NULL)
int fuse_argc = sizeof(fuse_argv)/sizeof(fuse_argv[0])-1;
// Запуск FUSE с заданными аргументами и операциями файловой системы
return fuse_main(fuse_argc, fuse_argv, &jsonfs_oper, NULL);
}

```